

**CONTROLLABLE MOTION-GUIDED IMAGE EDITING USING DIFFUSION
MODELS, MOTION PRIMITIVES, AND RESTORATION-AWARE
COMPOSITING**

by

Maham Mubasher

Supervisor

Dr. Ali Ahmed

A thesis submitted in partial fulfillment of the requirements for the degree of Master of
Science in Data Science.

Information Technology University (ITU)
Department of Science and Technology

2026

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Dr. Ali Ahmed, for his guidance, constructive feedback, and continuous support throughout the course of this thesis. His supervision helped me refine the research direction, strengthen the technical implementation, and present the work with clarity and academic rigor.

I am also grateful to the Department of Science and Technology at Information Technology University (ITU) for providing the academic environment and resources necessary to complete this research. I would like to acknowledge the broader research community whose contributions in diffusion models, optical flow, and motion-guided image editing provided the foundation for this work.

Finally, I extend my appreciation to my family, friends, and colleagues for their encouragement, patience, and support during the development and documentation of this thesis.

Table of Contents

Chapter	Title	Page
	Title Page	i
	Acknowledgments	ii
	Table of Contents	iii
	List of Figures	viii
	List of Tables	x
	Abstract	xi
1	Introduction	1
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Research Motivation	1
	1.4 Aim and Objectives	2
	1.5 Research Contributions	2
	1.6 Scope of the Study	3
	1.7 Thesis Organization	3
2	Literature Review	4
	2.1 Diffusion Models for Image Generation	4
	2.2 Guidance in Diffusion Sampling	4
	2.3 Pretrained Model Composition for Editing	4
	2.4 Diffusion-Based Image Editing	5
	2.5 Optical Flow and Motion-Guided Editing	5

	2.6 Point-Based and Interactive Editing	6
	2.7 Image Inpainting and Background Restoration	6
	2.8 Research Gap	6
3	Baseline Motion Guidance	8
	3.1 Baseline Overview	8
	3.2 Pretrained ML Components	8
	3.3 Motion Guidance Objective	8
	3.4 Flow-Based Guidance Loss	9
	3.5 Source Preservation	9
	3.6 Guidance During DDIM Sampling	9
	3.7 Baseline Reproduction	10
	3.8 Observed Baseline Limitations	10
4	Proposed Method	11
	4.1 Overview of the Proposed Pipeline	11
	4.2 Machine-Learning Model Stack	11
	4.3 Differentiable RAFT Guidance Energy	12
	4.4 Motion Primitive Parameterization	12
	4.5 Selective Refinement	13
	4.6 Hard Warp Initialization	13
	4.7 Guided DDIM Update	14
	4.8 Latent Preservation Outside the Edit Region	14
	4.9 Inference Diagnostics	15
	4.10 Background Restoration and Sharp Compositing	15
	4.11 Non-Evaluated Reference-Output Mode	16

5	ML Architecture and Implementation	17
	5.1 System Overview	17
	5.2 Backend Generation Pipeline	17
	5.3 Stable Diffusion Model Configuration	18
	5.4 RAFT as a Differentiable Loss Network	18
	5.5 Motion Primitive and Refinement Modules	19
	5.6 Modified Diffusion Sampler	19
	5.7 Latent-Space Preservation and Initialization	20
	5.8 ML Diagnostics and Saved Artifacts	20
	5.9 Hard Warp Initialization	20
	5.10 Restoration and Compositing Module	21
	5.11 Experiment Launcher and Visualization	21
	5.12 Implementation Summary	21
6	Experimental Setup	23
	6.1 Purpose of the Experiments	23
	6.2 Example Set	23
	6.3 Experimental Configurations	24
	6.4 Compared Method Variants	25
	6.5 Qualitative Evaluation Criteria	26
	6.6 Evidence Categories	27
	6.7 ML Environment and Validation	28
7	Results and Evaluation	29
	7.1 Evaluation Overview	29
	7.2 Implemented ML Behavior	29

	7.3 Apple Ablation Protocol	30
	7.4 Quantitative Results	30
	7.5 Diagnostic Trajectories	31
	7.6 Visual Results	31
	7.7 Answer to the RAFT Contribution Question	31
	7.8 Additional Generated Cases	35
	7.9 Status of Other Examples	35
	7.10 Evaluation Summary	35
8	Component Analysis	39
	8.1 Purpose of the Analysis	39
	8.2 Ablation Variants	39
	8.3 Contribution of Primitive Flow and Hard-Warp Initialization	39
	8.4 Contribution of RAFT-Guided Diffusion	40
	8.5 Contribution of Restoration and Compositing	40
	8.6 Component Findings	41
9	Limitations, Ethics, and Future Work	43
	9.1 Limitations	43
	9.2 Ethical Considerations	44
	9.3 Future Work	45
	9.4 Summary	46
10	Conclusion	47
	10.1 Conclusion	47
	10.2 Summary of Contributions	47

10.3 Main Findings	47
10.4 Answer to the Research Question	48
10.5 Final Remarks	48
References	50
References	50
Appendix A: Development and Weekly Progress Log	52
Appendix B: Commands, Parameters, and Reproducibility	56
Appendix C: Additional Result Images	63

List of Figures

Figure	Title	Page
5.1	Repository asset showing the apple motion-editing case used as the primary qualitative example.	18
7.1	Recorded flow-loss trajectories for the apple ablation. RAFT-guided primitive diffusion reaches lower flow loss than the no-RAFT variant across the three seeds.	32
7.2	Total guidance-energy trajectories for the apple ablation. RAFT-guided variants have lower final energy than the no-RAFT diffusion condition.	33
7.3	Final saved apple outputs. Rows are seeds 0–2. Columns are warp-inpaint-only, original Motion Guidance, diffusion without RAFT, diffusion with RAFT, and full pipeline. The full pipeline is visually strong, but its final image is composited.	33
7.4	Pre-composite apple outputs for primitive diffusion variants. Rows are seeds 0–2. Columns are no-RAFT diffusion, RAFT-guided diffusion, and full-pipeline pre-composite output. These images separate diffusion behavior from final compositing.	34
7.5	Additional teapot translation case over three seeds. Columns show source, warped initialization, decoded diffusion output, and final composited output.	36
7.6	Additional tree file-flow squeeze case over three seeds. This case is useful as a stress test; it should not be presented as a polished rigid-translation result.	37
10.1	Apple result asset used as the main qualitative evidence for controllable rigid object translation.	63
10.2	Final outputs from the apple ablation. This grid supports the qualitative comparison in Chapter 7.	65
10.3	Pre-composite outputs for primitive diffusion variants. Rows are seeds 0–2; columns are no-RAFT diffusion, RAFT-guided diffusion, and the full-pipeline pre-composite output.	66

10.4	Additional teapot translation runs over seeds 0–2. Columns show source, warped initialization, decoded diffusion output, and final composited output.	67
10.5	Additional tree file-flow squeeze runs over seeds 0–2. The case is useful for diagnosing non-rigid/file-flow behavior, not for claiming polished non-rigid editing.	68

List of Tables

Table	Title	Page
2.1	Summary of related methods and their relevance to this thesis.	7
3.1	Baseline Motion Guidance runs used for comparison.	10
5.1	Main implementation components.	22
6.1	Examples used in the experimental setup.	24
6.2	Pretrained ML components used by the editing pipeline.	24
6.3	Final experimental configurations.	25
6.4	Method variants evaluated in the thesis.	26
7.1	Apple ablation results averaged over three seeds. The deterministic baseline has no diffusion loss trajectory because sampling is skipped.	32
7.2	Additional generated cases averaged over three seeds. These results broaden the qualitative evidence but are not controlled component ablations.	36
7.3	Summary of examples and evidence status.	38
8.1	Controlled apple ablation variants.	42
10.1	Weekly development summary.	53
10.2	Implementation artifacts used in the thesis.	53
10.3	Final status of project examples.	55
10.4	Main reproducibility parameters.	62
10.5	Status of additional qualitative examples.	65

Abstract

This thesis investigates inference-time control of a pretrained latent diffusion model for single-image motion editing. The ML stack consists of Stable Diffusion v1.4, its AutoencoderKL image representation, a frozen CLIP text encoder, a conditional U-Net denoiser, and pretrained RAFT optical-flow estimation. Rather than training new weights, the method differentiates a motion-and-appearance energy through RAFT and the VAE decoder to modify the DDIM latent trajectory.

Translation, scale, stretch, and rotation parameters are converted into dense target flows that supervise the RAFT-based loss. At each denoising step, classifier-free text guidance predicts noise, the estimated clean latent is decoded, RAFT predicts source-to-output motion, and the flow and color losses are back-propagated to the latent. The implementation also uses mask-dependent gradient weights, gradient clipping, a guidance schedule, and RePaint-style latent replacement outside the edit region.

The evaluation reports a focused apple ablation with five method variants and three random seeds. The results show that RAFT-guided diffusion reduces final flow loss from 8.46 to 5.80 and reduces the best recorded flow loss from 8.16 to 3.07 compared with a no-RAFT diffusion variant. This provides measurable evidence that RAFT contributes to the pre-composite diffusion trajectory. However, the visually strongest final apple result is produced after deterministic restoration and source-object compositing, so the final image should be interpreted as a hybrid rendering result rather than an isolated diffusion output.

For large rigid translation, the system additionally encodes a geometrically initialized image into the diffusion latent space. Restoration and compositing address disoccluded background after sampling; for contained translations, the final saved image is therefore a hybrid result rather than a pure diffusion output. The contribution is an implemented and auditable inference-time ML pipeline with a limited but reproducible case-study ablation. Broader quantitative evaluation across additional objects, backgrounds, motion magnitudes, masks, and hyperparameters remains future work.

Chapter 1

Introduction

1.1 Background

Diffusion models have significantly improved image generation and editing by learning powerful image priors from large datasets. Stable Diffusion performs this generative process in a compressed latent space: a variational autoencoder maps images to latent tensors, a text-conditioned U-Net predicts diffusion noise, and a decoder maps denoised latents back to pixels. Although this model stack produces realistic images, text conditioning does not directly specify pixel-level object motion.

Motion-guided image editing addresses this limitation by adding a learned motion estimator to the diffusion loop. Motion Guidance uses RAFT, a pretrained optical-flow network, as a differentiable loss model during DDIM sampling [1, 2]. The current decoded image is compared with the source through predicted optical flow; the resulting loss is back-propagated through RAFT and the VAE decoder to the diffusion latent. This inference-time optimization is the central machine-learning mechanism studied in this thesis.

1.2 Problem Statement

The main technical problem addressed in this thesis is how to control a pretrained latent diffusion model with explicit spatial motion without retraining its parameters. In the original Motion Guidance workflow, a dense target flow supervises a differentiable RAFT loss. The thesis preserves this learned guidance mechanism but generates its supervision from geometric primitives and modifies how the resulting gradient is applied in latent space.

In addition, motion guidance alone may produce weak displacement, visible source-hole artifacts, or unstable background reconstruction after an object has been moved. Therefore, this thesis investigates the following question:

How can Motion Guidance be integrated with parameterized motion, spatial gradient weighting, geometric initialization, and restoration to support direct rigid-object editing?

1.3 Research Motivation

Precise object-level editing is useful in creative design, visual prototyping, and interactive generative tools. A practical system should allow the user to specify motion with understandable parameters

rather than relying only on dense flow files. For example, moving an apple to the right should be expressible through a displacement value, while the system should preserve the apple texture and repair the background region revealed by the movement.

This motivates an inference-time ML extension that combines a frozen Stable Diffusion model, frozen CLIP text encoder, frozen VAE, differentiable RAFT guidance, latent-space masking, gradient clipping, and a timestep-dependent guidance schedule. Geometric initialization and restoration support the ML pipeline where large displacement and disocclusion are difficult.

1.4 Aim and Objectives

The aim of this thesis is to develop and evaluate a controllable extension of Motion Guidance for single-image editing. The objectives are:

1. To reproduce the pretrained Stable Diffusion and RAFT-based Motion Guidance baseline.
2. To generate target flows from motion primitives such as translation, scale, stretch, and rotation.
3. To modify the latent DDIM trajectory using differentiable flow and color losses.
4. To apply different guidance-gradient weights inside and outside the edited region.
5. To use geometric initialization to enforce object displacement.
6. To reduce source-hole artifacts through restoration and sharp compositing.
7. To log flow loss, color loss, noise norms, and guidance-gradient norms during inference.
8. To evaluate the implemented pipeline through a qualitative case study, component analysis, and explicit limitation analysis.

1.5 Research Contributions

This thesis contributes an inference-time ML control pipeline built from pretrained generative and discriminative models. The main contributions are:

1. A primitive-to-flow module that produces dense supervision for the pretrained RAFT guidance network.
2. A modified DDIM loop that differentiates the motion energy through the VAE decoder and RAFT to update latent denoising.

3. Mask-dependent gradient weighting and RePaint-style latent replacement for spatially localized editing.
4. Latent initialization from a geometrically transformed image to study stronger starting conditions for large motion.
5. Inference diagnostics that record total guidance energy, flow loss, color loss, noise norms, and gradient norms.
6. Restoration and compositing for the disocclusion problem that remains after ML-guided motion.

1.6 Scope of the Study

This thesis focuses on single-image editing with pretrained Stable Diffusion v1.4 and pretrained RAFT. No network weights are trained or fine-tuned. The machine-learning contribution lies in model composition and inference-time optimization: neural image generation, neural text conditioning, neural optical-flow estimation, differentiable loss propagation, and latent trajectory control.

The empirical scope is narrow. The thesis package contains one primary controlled case study for rigid apple translation, supported by implementation traces, fixed-seed ablations, and diagnostic records. It also contains additional three-seed generated runs for a teapot translation and a tree file-flow squeeze case. These extra cases broaden the inspectable evidence, but they are not controlled component ablations and do not establish generalization. No claim of state-of-the-art performance or broad superiority is made.

1.7 Thesis Organization

Chapter 2 reviews related work. Chapter 3 explains the baseline Motion Guidance system. Chapter 4 presents the proposed method. Chapter 5 describes the implementation. Chapter 6 explains the experimental setup. Chapter 7 presents the available evidence. Chapter 8 gives the component analysis. Chapter 9 discusses limitations, ethics, and future work. Chapter 10 concludes the thesis.

Chapter 2

Literature Review

2.1 Diffusion Models for Image Generation

Diffusion models generate images by learning to reverse a gradual noising process. Denoising Diffusion Probabilistic Models (DDPMs) showed that high-quality image synthesis can be achieved through iterative denoising with a learned probabilistic model [3]. However, the original sampling process is computationally expensive because it requires many sequential denoising steps. Denoising Diffusion Implicit Models (DDIMs) improved sampling efficiency by introducing a non-Markovian sampling process that can use fewer steps while keeping the same training objective [4].

Latent Diffusion Models further improved efficiency by applying diffusion in the latent space of an autoencoder instead of directly in pixel space [5]. This design is important for practical image editing because it reduces computational cost while still supporting high-quality image synthesis and conditioning. The implementation used in this thesis builds on this family of latent diffusion methods.

2.2 Guidance in Diffusion Sampling

Guidance methods control the direction of the denoising process. Classifier-free guidance combines conditional and unconditional predictions to improve prompt alignment without requiring a separate classifier [6]. This is widely used in text-conditioned diffusion systems.

For image editing, guidance can also be introduced through external losses. In this setting, the diffusion sample is decoded during sampling, a task-specific loss is computed, and its gradient is used to steer the latent update. This idea is central to Motion Guidance, where the external loss is based on optical-flow consistency and source preservation [1]. The present thesis follows this direction but adds primitive control, localized refinement, and restoration-aware post-processing.

2.3 Pretrained Model Composition for Editing

The ML pipeline used in this thesis combines models with different learned functions. Stable Diffusion v1.4 supplies the generative prior through three principal components: an AutoencoderKL that compresses RGB images to four-channel latent tensors, a frozen CLIP text encoder that maps prompts to a 768-dimensional conditioning context, and a conditional U-Net that predicts noise at each diffusion timestep. The repository configuration uses a latent scaling factor of 0.18215, cross-attention conditioning, and a U-Net with spatial-transformer attention.

RAFT supplies a complementary learned capability. Instead of generating pixels, it estimates dense correspondence between the source image and the current decoded prediction. Because the flow estimator and VAE decoder are differentiable, motion error can be propagated back to the diffusion latent without updating the parameters of either network. RAFT evaluates whether the current sample follows the requested displacement, while Stable Diffusion constrains the update through its learned image prior.

The method is therefore an example of inference-time model guidance rather than supervised training or fine-tuning. Its network weights remain unchanged; optimization occurs over the current latent variable during sampling. The ML contribution lies in defining supervision, computing gradients through pretrained networks, and controlling the denoising trajectory.

2.4 Diffusion-Based Image Editing

Several diffusion-based editing methods focus on modifying an existing image while preserving its important content. SDEdit adds noise to an input image and then denoises it through a diffusion prior, allowing edits guided by user input without task-specific retraining [7]. Prompt-to-Prompt controls cross-attention maps to preserve layout while changing text prompts [8]. InstructPix2Pix trains a conditional diffusion model to follow natural-language image-editing instructions [9].

These methods demonstrate the strength of diffusion models for semantic and text-guided editing. However, text and instruction-based editing may still be imprecise for exact object displacement. A prompt such as “move the object to the right” does not directly define the object support, displacement magnitude, boundary handling, or background repair. This motivates motion-based control.

2.5 Optical Flow and Motion-Guided Editing

Optical flow estimates pixel-level motion between images. RAFT introduced a recurrent all-pairs field transform architecture that builds correlation volumes and iteratively refines flow estimates [2]. Its accuracy and differentiable design make it useful as a guidance signal in optimization and diffusion-based editing.

Motion Guidance uses a differentiable optical-flow estimator during diffusion sampling to encourage the generated image to match a user-provided target flow [1]. This enables dense motion-based image editing without training a new diffusion model. The limitation, however, is practical control: the user must often rely on precomputed flow files. This thesis addresses that limitation by generating target flows from explicit primitives such as translation, scale, stretch, and rotation.

2.6 Point-Based and Interactive Editing

Interactive editing methods provide another form of spatial control. DragGAN allows users to move image points on a GAN-generated image by optimizing generator features toward target locations [10]. DragDiffusion extends point-based interactive editing to diffusion models by optimizing latent features for handle-point movement [11]. These methods show the importance of direct spatial control, but they differ from this thesis because they use point manipulation rather than primitive-to-flow motion fields.

The proposed work is positioned between dense-flow motion guidance and interactive control. It keeps the motion-guided diffusion formulation but exposes simpler geometric controls that are easier to specify and evaluate.

2.7 Image Inpainting and Background Restoration

Moving an object creates a source-hole region at the original location. This requires background restoration in addition to object motion. RePaint uses a pretrained DDPM for inpainting by conditioning reverse diffusion on known image regions [12]. LaMa addresses large-mask inpainting using Fourier convolutions and a large effective receptive field, making it suitable for complex missing regions and high-resolution completion [13].

In this thesis, restoration is treated as a practical post-processing component rather than as the main generative contribution. The system combines local filling, patch-based restoration, OpenCV inpainting, and LaMa-based restoration to reduce visible source-hole artifacts after object translation.

2.8 Research Gap

The reviewed literature shows that diffusion models are strong generative priors, optical flow provides dense motion supervision, and inpainting methods can repair missing regions. However, a practical gap remains: motion-guided editing is powerful but difficult to control when the user must depend mainly on dense precomputed flows.

This thesis addresses the gap by integrating Motion Guidance with motion primitives, spatial gradient weighting, geometric initialization, and restoration-aware compositing. The contribution is not a new diffusion model; it is an implemented hybrid workflow built on existing motion-guided diffusion and inpainting components.

Table 2.1: Summary of related methods and their relevance to this thesis.

Method	Control Type	Main Strength	Limitation for This Thesis
DDPM / DDIM	Generative sampling	Strong image prior	Not designed for direct object motion
Latent Diffusion	Text and conditioning	Efficient high-quality synthesis	Spatial edits remain indirect
InstructPix2Pix / Prompt-to-Prompt	Text or instruction	Semantic editing	Limited precise motion control
DragGAN / DragDiffusion	Point dragging	Interactive spatial editing	Different from primitive flow control
Motion Guidance	Dense target flow	Motion-aware diffusion editing	Requires target flow specification
Proposed method	Motion primitives and restoration	Practical controllable translation	Strongest for rigid motion

Chapter 3

Baseline Motion Guidance

3.1 Baseline Overview

The baseline used in this thesis is the original Motion Guidance pipeline [1]. The method edits a source image by guiding a diffusion sampling process with a target optical-flow field. The input consists of a source image x_0 , a text prompt c , an edit mask M , and a target flow F . The output is an edited image that should follow the target motion while preserving the visual identity of the source image.

The baseline is important for two reasons. First, it provides the foundation on which the proposed method is built. Second, it defines the workflow against which the added interface and processing stages are conceptually compared.

3.2 Pretrained ML Components

The baseline is built on Stable Diffusion v1.4. Its AutoencoderKL encodes an RGB image into a four-channel latent representation and decodes predicted clean latents back to image space. A frozen CLIP ViT-L/14 text encoder produces a 768-dimensional prompt context, and a cross-attention U-Net predicts diffusion noise. The network parameters remain frozen during editing; the optimized quantity is the noisy latent z_t .

Motion supervision is supplied by pretrained RAFT. For each decoded clean-image estimate, RAFT predicts dense optical flow from the source image to the current output. Since the flow network and VAE decoder remain inside the computational graph, prediction error can be differentiated with respect to z_t . The baseline therefore couples a learned generative prior with a learned motion estimator at inference time.

3.3 Motion Guidance Objective

Motion Guidance treats editing as a guided diffusion problem rather than as a separate training task. During sampling, the current image estimate is encouraged to satisfy a motion-consistency objective. In simplified form, the baseline guidance energy can be written as:

$$E(x) = \lambda_f L_{\text{flow}}(x, x_0, F) + \lambda_c L_{\text{color}}(x, x_0), \quad (\text{Equation 3.1})$$

where L_{flow} encourages the estimated motion between the source image and edited image to match

the target flow, and L_{color} encourages visual consistency with the source image. The weights λ_f and λ_c control the relative strength of motion consistency and source preservation.

3.4 Flow-Based Guidance Loss

The baseline uses RAFT as a differentiable optical-flow estimator [2]. Given the source image x_0 and the current decoded image x , RAFT estimates a flow field:

$$\hat{F} = \Phi(x_0, x), \quad (\text{Equation 3.2})$$

where Φ denotes the optical-flow network. The implemented flow loss is the mean absolute difference between the predicted and target flows:

$$L_{\text{flow}} = \text{MAE}(\hat{F}, F). \quad (\text{Equation 3.3})$$

This loss provides the main motion signal. If the generated image does not follow the intended displacement or deformation, the flow loss increases and its gradient is used to steer the diffusion sampling process.

3.5 Source Preservation

Motion consistency alone is not sufficient because an image can satisfy a motion field while still changing object or background appearance. The implementation backward-warps the generated image using the predicted flow, or the target flow when oracle mode is active, and compares it with the source:

$$L_{\text{color}} = \text{MAE}(x_0, W(x, F_{\text{warp}})), \quad (\text{Equation 3.4})$$

where $W(\cdot)$ denotes differentiable warping. Optional occlusion masking excludes unreliable disoccluded comparisons. This term encourages appearance preservation while the RAFT loss supplies motion supervision.

3.6 Guidance During DDIM Sampling

The baseline uses a modified DDIM sampler. At each denoising step, the current latent estimate is decoded into an image estimate. The motion-guidance energy is then computed on this decoded estimate, and the gradient of the energy is back-propagated to the latent variable:

Table 3.1: Baseline Motion Guidance runs used for comparison.

Example	Prompt	Mask	Target Flow	Steps
Apple	an apple on a wooden table	right mask	right flow	40
Teapot	a teapot floating in water	down mask	down flow	40
Topiary	a photo of topiary	mask.pth	flow.pth	40

$$g_t = \nabla_{z_t} E(\hat{x}_0). \quad (\text{Equation 3.5})$$

The predicted noise estimate is corrected using this gradient before the next DDIM update. Classifier-free guidance retains prompt conditioning, while RAFT supplies spatial supervision. The implementation records total energy, flow loss, color loss, U-Net noise norm, and guidance-gradient norm at each denoising step.

3.7 Baseline Reproduction

The baseline was reproduced using the original file-flow mode. In this mode, the target motion is loaded from a precomputed flow file rather than generated from user-defined primitives. Table 3.1 summarizes the baseline runs used as the starting comparison point.

The baseline runs used guidance weight 30.0, gradient clipping value 60.0, one recursive step, one RAFT iteration, and classifier-free guidance scale 7.5. These settings established a stable reference point before adding thesis-specific components.

3.8 Observed Baseline Limitations

The reproduced baseline confirmed that Motion Guidance is a strong foundation for motion-aware editing, but it also exposed practical limitations relevant to this thesis.

First, the baseline depends on precomputed dense target flows. This makes the workflow less intuitive for users who want to specify simple motion such as “move right by 150 pixels” or “stretch vertically”. Second, soft guidance alone may not produce sufficiently strong object displacement for primitive-style control. Third, after an object is moved, the original object location may contain source-hole artifacts that require restoration.

These limitations motivate the proposed extension in Chapter 4. The thesis contribution is therefore not the original Motion Guidance model, but a hybrid pipeline built around primitive flow generation, spatial guidance weighting, geometric initialization, and restoration-aware compositing.

Chapter 4

Proposed Method

4.1 Overview of the Proposed Pipeline

The proposed method extends Motion Guidance as an inference-time ML pipeline. Instead of training or fine-tuning model weights, it controls the latent variable of pretrained Stable Diffusion v1.4. A geometric primitive defines target motion, pretrained RAFT evaluates motion consistency on each decoded prediction, and gradients are propagated through RAFT and the VAE decoder to alter DDIM denoising. Geometric initialization and restoration address failure modes that are difficult for the learned guidance alone.

Given a source image $x_0 \in [0, 1]^{H \times W \times 3}$, text condition c , object support mask M , and target flow $F \in \mathbb{R}^{H \times W \times 2}$, the external guidance energy is:

$$E(x) = \lambda_f L_{\text{flow}}(x, x_0, F) + \lambda_c L_{\text{color}}(x, x_0), \quad (\text{Equation 4.1})$$

where L_{flow} enforces motion consistency and L_{color} encourages source appearance preservation. The text-conditioned diffusion model supplies the image prior through its denoising prediction; it is not an additional term returned by the implemented guidance-energy function. The gradient of Equation Equation 4.1 is applied during sampling.

4.2 Machine-Learning Model Stack

The method composes four pretrained neural components. First, the AutoencoderKL maps an image x to a four-channel latent $z = 0.18215 E_{\text{VAE}}(x)$ and decodes latent predictions to RGB. Second, a frozen CLIP ViT-L/14 encoder maps the text prompt to conditioning vector c . Third, the Stable Diffusion U-Net $\epsilon_\phi(z_t, t, c)$ predicts noise under cross-attention conditioning. Fourth, RAFT $\Phi(x_0, x)$ predicts dense optical flow between the source and current decoded image.

All model parameters are held fixed. The variable updated during generation is z_t . The computational path for ML guidance is

$$z_t \rightarrow \hat{z}_0 \rightarrow D_{\text{VAE}}(\hat{z}_0) \rightarrow \Phi(x_0, \hat{x}_0) \rightarrow E(\hat{x}_0), \quad (\text{Equation 4.2})$$

followed by automatic differentiation from E back to z_t . This allows a pretrained discriminative model, RAFT, to supervise a pretrained generative model during inference.

4.3 Differentiable RAFT Guidance Energy

For the decoded estimate $\hat{x}_0$, RAFT predicts

$$\hat{F}_t = \Phi(x_0, \hat{x}_0). \quad (\text{Equation 4.3})$$

The implemented motion loss is

$$L_{\text{flow}} = \text{MAE}(\hat{F}_t, F). \quad (\text{Equation 4.4})$$

To preserve appearance, the prediction is backward-warped and compared with the source:

$$L_{\text{color}} = \text{MAE}(x_0, W(\hat{x}_0, F_{\text{warp}})). \quad (\text{Equation 4.5})$$

For primitive mode, the implementation enables oracle warping, so $F_{\text{warp}} = F$; otherwise the RAFT prediction is used. An optional occlusion mask suppresses comparisons in disoccluded regions. This energy supplies supervision without requiring ground-truth edited images.

4.4 Motion Primitive Parameterization

The first contribution is to generate target motion fields from explicit primitives. Let $p = (x, y)^\top$ denote a pixel coordinate and let $T_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ be a geometric transformation with parameters θ . The induced flow is:

$$F_\theta(p) = T_\theta(p) - p. \quad (\text{Equation 4.6})$$

The flow is restricted to the object support:

$$F(p) = \begin{cases} F_\theta(p), & p \in M, \\ (0, 0)^\top, & p \notin M. \end{cases} \quad (\text{Equation 4.7})$$

For translation with horizontal and vertical displacement (d_x, d_y) :

$$T_{\text{trans}}(p) = p + \begin{bmatrix} d_x \\ d_y \end{bmatrix}, \quad F_{\text{trans}}(p) = \begin{bmatrix} d_x \\ d_y \end{bmatrix}. \quad (\text{Equation 4.8})$$

For scaling around object center $c_m = (c_x, c_y)^\top$:

$$T_{\text{scale}}(p) = c_m + \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} (p - c_m), \quad (\text{Equation 4.9})$$

where s_x and s_y are horizontal and vertical scale factors. Stretching is implemented as anisotropic scaling, using different values for s_x and s_y .

For rotation by angle α :

$$T_{\text{rot}}(p) = c_m + \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix} (p - c_m). \quad (\text{Equation 4.10})$$

These primitives provide direct user control while still producing a dense flow field compatible with the original Motion Guidance loss.

4.5 Selective Refinement

The second component is spatial guidance weighting, called selective refinement in the implementation. The baseline gradient is global, so the proposed system multiplies it by different weights inside and outside the editable region. This is a simple weighting rule rather than a learned refinement model.

Let $g_t = \nabla_{z_t} E(\hat{x}_0)$ be the guidance gradient at diffusion step t , and let M_z be the edit mask resized to latent resolution. A spatial weight map is defined as:

$$w(p) = \begin{cases} w_{\text{in}}, & p \in M_z, \\ w_{\text{out}}, & p \notin M_z. \end{cases} \quad (\text{Equation 4.11})$$

The refined gradient is:

$$\tilde{g}_t(p) = w(p)g_t(p). \quad (\text{Equation 4.12})$$

In the reported configuration, w_{in} is larger than w_{out} . This localizes the applied gradient by construction. The available evidence does not isolate its effect quantitatively, so source-preservation improvement is treated as a design intention rather than a measured result.

4.6 Hard Warp Initialization

Primitive flow used only as a soft guidance signal was not sufficient for reliable object movement. To address this, the proposed method adds hard warp initialization. The source image is first geometrically moved according to the target flow before diffusion refinement begins:

$$x_{\text{warp}}(p) = x_0(p - F(p)). \quad (\text{Equation 4.13})$$

The warped image is then encoded into the latent space:

$$z_{\text{init}} = \gamma E_{\text{VAE}}(x_{\text{warp}}), \quad (\text{Equation 4.14})$$

where E_{VAE} is the encoder of the latent diffusion model and γ is the latent scaling factor used by the model. This gives the diffusion sampler a stronger initial condition because the object is already placed near the desired location before denoising refinement.

4.7 Guided DDIM Update

During sampling, classifier-free guidance first computes a text-guided noise estimate:

$$\epsilon_{\text{cfg}} = \epsilon_{\phi}(z_t, t, \emptyset) + s(\epsilon_{\phi}(z_t, t, c) - \epsilon_{\phi}(z_t, t, \emptyset)), \quad (\text{Equation 4.15})$$

where s is the classifier-free guidance scale. A clean-image estimate is computed from the current latent:

$$\hat{x}_0 = D_{\text{VAE}}\left(\frac{z_t - \sqrt{1 - \alpha_t}\epsilon_{\text{cfg}}}{\sqrt{\alpha_t}}\right). \quad (\text{Equation 4.16})$$

The motion-guidance gradient is then computed as:

$$g_t = \nabla_{z_t} E(\hat{x}_0). \quad (\text{Equation 4.17})$$

After selective refinement and gradient clipping, the noise estimate is corrected:

$$\epsilon'_t = \epsilon_{\text{cfg}} - \sqrt{1 - \alpha_t} \eta_t \tilde{g}_t, \quad (\text{Equation 4.18})$$

where η_t represents the guidance weight and schedule at timestep t . The corrected noise estimate is then used in the DDIM update. This keeps the diffusion prior active while steering the edit toward the desired motion.

The default implementation applies guidance during the first 80% of DDIM steps and sets it to zero for the final 20%. This allows late denoising steps to consolidate image detail without continued motion pressure. The gradient norm is clipped when it exceeds the configured threshold, limiting unstable latent updates caused by large RAFT or reconstruction gradients.

4.8 Latent Preservation Outside the Edit Region

The sampler preserves non-edited regions through latent replacement. At timestep t , positions outside the editable support are replaced with either cached inversion latents or a forward-noised source latent:

$$z_t(p) \leftarrow z_t^{\text{source}}(p), \quad p \notin M_z. \quad (\text{Equation 4.19})$$

This operation is related to RePaint-style conditioning: the diffusion model remains free to update the editable region while source-consistent latent values are reintroduced elsewhere. If cached latents do not match the active DDIM step count, the implementation reindexes them; if too few are available, it falls back to forward-process source latents.

4.9 Inference Diagnostics

The sampler stores five trajectories for each run: total guidance energy, RAFT flow loss, color-preservation loss, U-Net noise norm, and guidance-gradient norm. It can also save intermediate decoded reconstructions and RAFT flow visualizations. These artifacts expose the behavior of the ML optimization and provide a basis for future convergence analysis, even though the current thesis package does not contain a complete controlled metric study.

4.10 Background Restoration and Sharp Compositing

When an object is translated, the original object location becomes a source-hole region. Let S be the source object support and S' be the shifted support. The source-hole mask is:

$$H = S(1 - S'). \quad (\text{Equation 4.20})$$

The background is restored using an inpainting or patch-based operator:

$$B = \mathcal{I}(x_0, H), \quad (\text{Equation 4.21})$$

where $\mathcal{I}$ denotes the selected restoration method. In practice, the implementation supports local filling, patch-based restoration, OpenCV inpainting, and LaMa-based restoration.

The shifted object is:

$$O'(p) = S'(p)x_0(p - F(p)). \quad (\text{Equation 4.22})$$

For contained rigid translations, the final saved image is:

$$x_{\text{final}}(p) = A(p)O'(p) + (1 - A(p))B(p), \quad (\text{Equation 4.23})$$

where A is a softened alpha mask derived from the shifted support. In the current implementation this compositing operation replaces the decoded diffusion sample for contained translated objects. It

preserves source-object texture, but it also means that the final apple-style result cannot be attributed solely to diffusion refinement.

4.11 Non-Evaluated Reference-Output Mode

The application contains an option that loads shipped output assets for selected presets. Because this path substitutes an existing image for the generated sample, it is not an algorithmic stabilization method and is excluded from evaluation. It is retained only as a user-interface demonstration feature. Non-rigid presets that depend on this option do not support claims about the proposed editing method.

Chapter 5

ML Architecture and Implementation

5.1 System Overview

The implementation is centered on inference-time interaction between pretrained ML models. Stable Diffusion v1.4 generates and refines image latents, CLIP supplies text conditioning, RAFT measures motion, and automatic differentiation converts the RAFT-based energy into a latent guidance signal. Primitive-flow generation, latent masking, geometric initialization, and restoration support this core ML loop.

The code is organized so that each ML responsibility is visible: model loading, target supervision, loss construction, guided sampling, latent preservation, diagnostics, and output restoration. The user interface is only a launcher for these experiments and is not treated as a research contribution.

5.2 Backend Generation Pipeline

The main generation pipeline is implemented in `backend/generate.py`. This file parses the experiment configuration, loads the source image, mask, target flow or primitive parameters, initializes the diffusion model and RAFT-based guidance loss, runs DDIM sampling, and saves the final output. The input configuration contains the following main elements:

- source image directory;
- edit mask file;
- target flow file or primitive flow mode;
- text prompt;
- guidance weight, clipping value, DDIM steps, and RAFT iterations;
- optional hard warp initialization;
- optional spatial guidance weighting;
- optional preservation and presentation-asset modes.

In file-flow mode, the target flow is loaded directly from the input dataset. In primitive mode, the target flow is generated from user-defined geometric parameters and saved as an output artifact. This design keeps the original Motion Guidance behavior available while adding the proposed controllable extension.

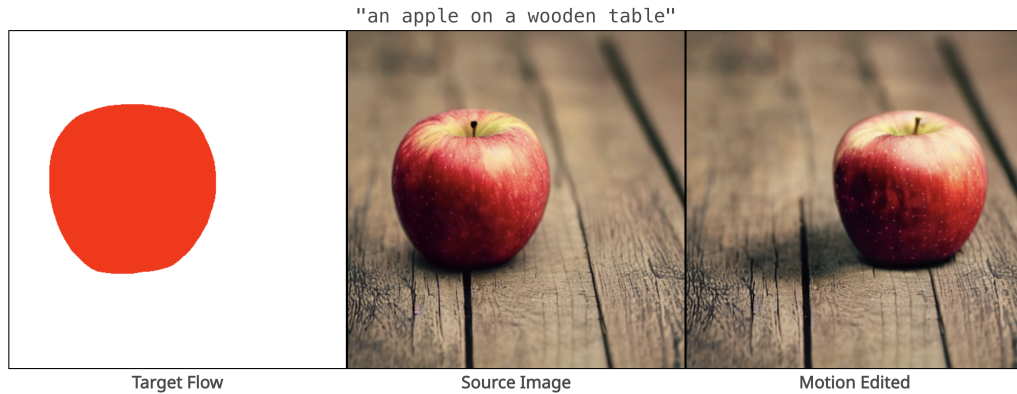


Figure 5.1: Repository asset showing the apple motion-editing case used as the primary qualitative example.

5.3 Stable Diffusion Model Configuration

The implementation loads the Stable Diffusion v1.4 checkpoint with the repository’s latent-diffusion configuration. The model contains:

- an AutoencoderKL with four latent channels for image encoding and decoding;
- a U-Net with base width 320, residual blocks, spatial transformers, and cross-attention;
- a frozen CLIP ViT-L/14 text encoder with context dimension 768;
- a latent scaling factor of 0.18215 and a diffusion process defined over 1000 training timesteps.

The checkpoint is loaded in evaluation mode. No optimizer is created and no model parameter is updated. Conditional and unconditional CLIP embeddings are computed once per prompt and reused by classifier-free guidance throughout sampling.

5.4 RAFT as a Differentiable Loss Network

The class `backend/losses.py:FlowLoss` instantiates RAFT and evaluates every decoded clean-image estimate against the source. The network returns dense flow, which is compared with the target using mean absolute error. A second loss backward-warps the current prediction and compares its colors with the source. The weighted sum is returned as a scalar PyTorch tensor, preserving the graph required for differentiation to the diffusion latent.

In primitive mode, the generated target flow is supplied directly to the loss and oracle warping is enabled for the color term. This avoids using RAFT’s current prediction as the appearance-warp target while RAFT still provides the learned flow estimate used by the motion loss.

5.5 Motion Primitive and Refinement Modules

The primitive motion logic is isolated in `backend/motion_primitives.py`. The implementation defines a compact primitive specification containing the primitive type and its parameters: translation offsets, scale factors, and rotation angle. The module implements translation, scale, stretch, and rotation by constructing a dense target-flow tensor over the object support.

The selective refinement logic is implemented in `backend/selective_refinement.py`. This module creates a latent-space weight map from the edit region and scales the guidance gradient using separate inner and outer weights. This design keeps the refinement step simple and explicit:

$$\tilde{g}_t = W_M \odot g_t, \quad (\text{Equation 5.1})$$

where g_t is the original motion-guidance gradient and W_M is the mask-dependent weight map. In implementation terms, the module is intentionally small so that the contribution is easy to audit and explain.

5.6 Modified Diffusion Sampler

The modified sampler is implemented in `backend/ldm/models/diffusion/ddim_with_grad.py`. At each denoising step, the sampler decodes an estimate of the clean image, computes the motion-guidance energy, back-propagates the energy to the latent variable, and modifies the predicted noise estimate before the DDIM update.

The sampler also supports five ML controls that are important for this thesis:

- cached latents for preserving regions outside the edit mask;
- gradient clipping for numerical stability;
- spatial weighting of the guidance gradient;
- timestep-dependent guidance that is disabled during the final 20% of steps;
- recursive denoising and logging of loss and norm trajectories.

This sampler is the bridge between the mathematical objective and the actual generated image. The proposed method does not train a new diffusion model; it changes how the existing diffusion model is guided during sampling.

5.7 Latent-Space Preservation and Initialization

Before each gradient computation, the sampler replaces latent values outside the editable region with cached source latents. When cached latents are missing, it samples the corresponding forward-diffusion distribution from the encoded source image. This prevents the U-Net and external guidance from freely rewriting the complete scene.

If geometric initialization is enabled, the transformed image is encoded by the same AutoencoderKL and mixed with noise at the initial DDIM level. When an inversion latent is available, its inferred noise component is reused. This gives the ML sampler a source-related stochastic state while changing the clean latent around which denoising begins.

5.8 ML Diagnostics and Saved Artifacts

Every run saves arrays for total guidance energy, RAFT flow loss, color loss, U-Net noise norm, and guidance-gradient norm. At a configured logging frequency, it also saves decoded clean-image estimates and visualizations of RAFT’s predicted flow. These artifacts are more informative for ML analysis than the final image alone because they reveal whether motion error decreases, whether appearance loss dominates, and whether gradient clipping is repeatedly activated.

The final experiment code adds two analysis utilities. The metric script reads saved run manifests, final images, pre-composite images, source images, and target flows to produce `metrics.csv`. The diagnostic plotting script reads the saved NumPy trajectories and produces combined flow-loss, total-energy, color-loss, noise-norm, and guidance-norm plots. These scripts make the RAFT contribution auditable rather than relying only on final images.

5.9 Hard Warp Initialization

Hard warp initialization is implemented in the generation pipeline before sampling begins. For primitive translation, the system constructs separate layers: restored background, shifted object, shifted support, and source-hole mask. The shifted image is used as a stronger initialization for the diffusion process.

For general file-flow cases, the source image is warped using the target flow. For primitive translation, the implementation uses an explicit layer construction because it gives better control over object placement and source-hole handling. This design is important because the experiments showed that primitive flow alone was too weak, while hard warp initialization made object movement operational.

5.10 Restoration and Compositing Module

Background restoration and compositing are implemented in the restoration module. This module was added because object translation creates a missing background region at the old object location. It supports several strategies:

- local masked filling;
- directional texture filling;
- patch-based restoration;
- OpenCV Telea inpainting;
- optional LaMa-based inpainting through `simple-lama-inpainting`.

The final contained-translation path separates background completion from object rendering. The background is restored first, and the original object is shifted and composited into its new location. This deterministic post-processing path preserves source texture, but it supersedes the decoded diffusion sample in the final saved image. The key operation is:

$$x_{\text{out}} = \alpha O' + (1 - \alpha)B, \quad (\text{Equation 5.2})$$

where O' is the shifted object, B is the restored background, and α is a softened support mask. This step is used by the contained rigid-translation path.

5.11 Experiment Launcher and Visualization

A FastAPI service and Next.js page expose the ML configuration, launch the Python generation process, report denoising progress, and display source and output images. These components improve repeatability but do not alter the model, loss, or sampling algorithm. Presentation modes that load shipped assets are excluded from evaluation.

5.12 Implementation Summary

Table 5.1 summarizes the main implemented components and their roles in the thesis.

The final implementation is primarily an ML inference system. Its core operation is the differentiable coupling of Stable Diffusion and RAFT; geometric initialization and restoration are auxiliary stages used for large rigid translation and disocclusion handling.

Table 5.1: Main implementation components.

Component	File	Role
Generation pipeline	backend/generate.py	Loads inputs, builds target flow, runs sampling, saves outputs.
Motion primitives	backend/motion_primitives.py	Converts primitive parameters into dense target flows.
Selective refinement	backend/selective_refinement.py	Applies mask-dependent guidance weights.
DDIM sampler	ddim_with_grad.py	Injects motion-guidance gradients during sampling.
Restoration	background_restoration.py	Repairs source holes and composites shifted objects.
Ablation command generator	experiments/run_core_ablation.py	Prints the five apple ablation commands for each seed.
Metric computation	experiments/compute_metrics.py	Produces CSV and JSON summaries from saved outputs.
Diagnostic plotting	experiments/plot_diagnostics.py	Plots saved loss and gradient-norm trajectories.
Experiment launcher	backend/mg_api/main.py	Validates inputs and launches ML inference jobs.
Visualization UI	frontend/src/app/page.tsx	Exposes parameters and displays outputs.

Chapter 6

Experimental Setup

6.1 Purpose of the Experiments

The experiments were designed as a case study of inference-time ML control. They examine how a pretrained latent diffusion model is conditioned by text, source latents, and gradients from a pretrained optical-flow model. Geometric initialization and restoration are evaluated as supporting stages around this ML process. The study does not establish broad statistical superiority, but it does include a controlled fixed-seed apple ablation over five variants and three seeds.

The evaluation therefore follows a staged structure:

1. reproduce Stable Diffusion inference with RAFT-based Motion Guidance;
2. replace file-only supervision with primitive-generated target flow;
3. test latent gradient weighting, clipping, and geometric initialization;
4. inspect saved flow loss, color loss, noise norm, and guidance norm trajectories;
5. add source-hole restoration and sharp compositing;
6. document which outputs are generated by the pipeline and which presets only display shipped assets.

6.2 Example Set

The experiments use six main examples from the Motion Guidance-style dataset and repository assets: apple, teapot, lion, tree, woman, and topiary. These examples were selected because they cover both rigid translation and more difficult deformation cases.

Only the apple example is used for the final metric table because it has the complete local input and output artifacts needed for repeated evaluation. Teapot, lion, and tree are documented configurations, but their complete repeated outputs are not part of the final ablation. Woman and topiary can load shipped reference assets and are therefore excluded from evaluation.

Table 6.1: Examples used in the experimental setup.

Example	Input Directory	Motion Type	Role in Evaluation
Apple	./data/apple	Translation	Included qualitative case study
Teapot	./data/teapot	Translation	Documented; output not included
Lion	./data/lion	Translation	Documented; output not included
Tree	./data/tree	Stretch	Documented; output not included
Woman	./data/woman	Non-rigid flow	Presentation preset; not evaluated
Topiary	./data/topiary	Non-rigid flow	Presentation preset; not evaluated

Table 6.2: Pretrained ML components used by the editing pipeline.

Component	Model	Function During Inference
Image representation	AutoencoderKL	Encodes RGB images to four-channel latents and decodes clean-latent estimates.
Text conditioning	CLIP ViT-L/14	Produces frozen prompt embeddings for cross-attention.
Generative prior	Stable Diffusion v1.4 U-Net	Predicts conditional and unconditional diffusion noise.
Motion estimator	RAFT	Predicts dense source-to-output flow inside the differentiable guidance loss.
Learned restoration	LaMa, optional	Inpaints the source-hole mask after object translation.

6.3 Experimental Configurations

6.3.1 Pretrained Models

Table 6.2 summarizes the learned models used during inference. No component is fine-tuned in this thesis.

Table 6.3 summarizes the main configurations used in the final experimental setup. The target-flow file is still listed for primitive examples because it is used to localize the object support, while the actual displacement is generated from primitive parameters.

The documented inference settings were:

- DDIM steps: 120;
- classifier-free guidance scale: 7.5;
- motion guidance weight: 30;

Table 6.3: Final experimental configurations.

Example	Mask	Flow Support	Mode	Key Parameters
Apple	right mask	right flow	Primitive translate	$d_x = 150, d_y = 0, w_{out} = 0.15$
Teapot	down mask	down flow	Primitive translate	$d_x = 0, d_y = 150, w_{out} = 0.15$
Lion	left mask	left flow	Primitive translate	$d_x = -100, d_y = 0, w_{out} = 0.15$
Tree	squeeze mask	squeeze flow	Primitive stretch	$s_x = 0.8, s_y = 1.1$
Woman	hair mask	hair flow	Presentation asset mode	Excluded from evaluation
Topiary	object mask	object flow	Presentation asset mode	Excluded from evaluation

- gradient clipping value: 60;
- recursive steps: 1;
- RAFT iterations: 1;
- precision: fp32.

These values record the final apple ablation configuration. Because the study does not include a controlled hyperparameter search, they should not be interpreted as optimal settings.

6.3.2 Recorded ML Diagnostics

For every denoising step, the modified sampler records total guidance energy, RAFT flow loss, color-preservation loss, the norm of the U-Net noise prediction, and the norm of the applied guidance update. Intermediate decoded predictions and RAFT flow visualizations can also be saved every five steps. These diagnostics are intended to answer ML-specific questions: whether predicted motion approaches the target, whether appearance preservation dominates the energy, and whether the external gradient remains numerically stable.

6.4 Compared Method Variants

The evaluation compares the method in stages rather than only showing the final output. This is important because the thesis contribution is cumulative and because final compositing can hide the contribution of diffusion. The final apple ablation uses the following five variants:

This staged description supports the component analysis in Chapter 8.

Table 6.4: Method variants evaluated in the thesis.

Variant	Description
Warp-inpaint-only	Diffusion is skipped; primitive translation, source-hole restoration, and final compositing are applied deterministically.
Original Motion Guidance	Original file-flow Motion Guidance baseline using the prepared target flow.
Diffusion, no RAFT	Primitive hard-warp initialization and preservation are used, but the RAFT guidance weight is zero.
Diffusion + RAFT	Primitive hard-warp initialization and preservation are used with RAFT motion guidance and no final composite.
Full pipeline	Primitive initialization, RAFT guidance, preservation, restoration, and final compositing are all enabled.

6.5 Qualitative Evaluation Criteria

The available thesis artifacts support both qualitative inspection and a focused quantitative apple ablation. The following criteria are used to interpret the saved metrics.

6.5.1 Motion Consistency

Motion consistency measures whether the estimated motion between the source image and edited image follows the target flow. Let $\Phi(x_0, x)$ be the RAFT-estimated flow between the source and output. The motion error is:

$$\mathcal{E}_{motion} = \frac{1}{|M|} \sum_{p \in M} \|\Phi(x_0, x)(p) - F(p)\|_1. \quad (\text{Equation 6.1})$$

Lower motion error indicates that the output better follows the requested target motion.

6.5.2 Source Preservation

Source preservation measures how much the output changes outside the edited region. For complement mask $(1 - M)$:

$$\mathcal{E}_{preserve} = \frac{1}{|\bar{M}|} \sum_{p \notin M} \|x(p) - x_0(p)\|_1. \quad (\text{Equation 6.2})$$

Lower preservation error means that the unedited background remains closer to the source image.

6.5.3 Object Sharpness

For translated objects, the object should remain sharp after movement. Sharpness is measured using the variance of the Laplacian inside the shifted support S' :

$$\mathcal{S}_{sharp} = \text{Var} \left(\nabla^2 x(p) \right), \quad p \in S'. \quad (\text{Equation 6.3})$$

Higher values generally indicate stronger high-frequency detail and less blur.

6.5.4 Source-Hole Plausibility

The source-hole region H is the area exposed after the object moves. The evaluation checks whether this region becomes plausible background rather than retaining a ghost of the original object. A simple source-hole artifact score can be computed as:

$$\mathcal{E}_{hole} = \frac{1}{|H|} \sum_{p \in H} \|x(p) - B(p)\|_1, \quad (\text{Equation 6.4})$$

where B is the restored or expected background estimate. In the implemented metric script, related source-hole and boundary scores are computed using local gradient and color differences around the exposed region.

6.5.5 Boundary Consistency

Boundary consistency evaluates whether the object boundary contains visible seams. Let $\partial S'$ denote a narrow band around the shifted support boundary. A boundary seam score can be defined using gradient differences:

$$\mathcal{E}_{boundary} = \frac{1}{|\partial S'|} \sum_{p \in \partial S'} \|\nabla x(p) - \nabla B(p)\|_1. \quad (\text{Equation 6.5})$$

Lower values indicate smoother transitions between the composited object and the restored background.

6.6 Evidence Categories

To avoid overclaiming, outputs are separated into two categories:

- **Evaluated pipeline output:** the apple ablation, consisting of five variants and three seeds.

- **Documented configurations:** teapot, lion, and tree settings recorded in development notes without complete local result artifacts.
- **Shipped presentation assets:** woman and topiary images loaded by an optional application mode; these are not experimental results.

This separation prevents interface demonstrations and incomplete records from being counted as empirical evidence.

6.7 ML Environment and Validation

The ML implementation uses Python, PyTorch, Stable Diffusion v1.4, RAFT, and optional LaMa inpainting. PyTorch autograd is required because the guidance energy must remain differentiable from RAFT’s output to the diffusion latent. The core ablation commands are generated with:

```
python experiments/run_core_ablation.py \
  --results-root results/core_ablation \
  --seed <0|1|2> \
  --ddim-steps 120 \
  --raft-iters 1
```

Metrics and plots are regenerated with:

```
python experiments/compute_metrics.py \
  --results-root results/core_ablation \
  --csv results/core_ablation/metrics.csv \
  --json results/core_ablation/metrics.json

python experiments/plot_diagnostics.py \
  --results-root results/core_ablation \
  --output-dir results/core_ablation/diagnostic_plots
```

Runtime validation checks the Stable Diffusion checkpoint, RAFT checkpoint, input image, mask, target flow, inversion latent, and cached latents. The API and frontend are optional launch and visualization utilities; they are not part of the ML method.

Chapter 7

Results and Evaluation

7.1 Evaluation Overview

This chapter reports the final evidence collected for the inference-time motion-editing pipeline. The evaluation is centered on the apple translation example because it is the only example for which the complete source image, mask, flow support, cached latents, generated outputs, diagnostic arrays, and fixed-seed repeated runs are locally available. The experiment therefore supports a focused case study rather than a broad benchmark across object categories.

The final apple evaluation contains five method variants and three random seeds, giving fifteen runs in total. The variants are designed to answer the central examination question: if the final apple is produced by geometric movement, inpainting, and source-object compositing, what measurable contribution does RAFT-guided diffusion make?

7.2 Implemented ML Behavior

The implementation confirms that the pretrained models are coupled in one differentiable inference graph. At each DDIM step, the U-Net produces conditional and unconditional noise estimates, classifier-free guidance combines them, the VAE decodes a clean-image estimate, and RAFT predicts dense motion relative to the source. PyTorch autograd then differentiates the weighted flow and color energy with respect to the current latent. No Stable Diffusion, CLIP, VAE, or RAFT parameter is updated.

The sampler also preserves source information outside the edit region by replacing latent values with cached or forward-noised source latents. Spatial gradient weighting changes the strength of the external ML signal inside and outside the editable support, while gradient clipping bounds unusually large updates. These mechanisms operate directly on the inference trajectory rather than on the final pixels.

For contained translation, the implementation saves three image states: the decoded diffusion output, the pre-composite output after preservation, and the final output. This separation is important because the final full-pipeline image may use deterministic source-object compositing.

7.3 Apple Ablation Protocol

The apple experiment uses the prompt *an apple on a wooden table*, the source folder `./data/apple`, the rightward object support, and a primitive translation of $d_x = 150$, $d_y = 0$. Each stochastic diffusion variant is run with seeds 0, 1, and 2. The main settings are 120 DDIM steps, classifier-free guidance scale 7.5, guidance weight 30 for RAFT-guided variants, gradient clipping at 60, one recursive step, one RAFT iteration, and fp32 precision.

The five evaluated variants are:

1. **warp-inpaint-only:** deterministic primitive translation, source-hole restoration, and final compositing, with diffusion sampling skipped;
2. **original Motion Guidance:** file-based dense-flow guidance without the new primitive initialization and without final compositing;
3. **diffusion without RAFT guidance:** primitive hard-warp initialization, source preservation, and zero motion-guidance weight;
4. **diffusion with RAFT guidance:** primitive hard-warp initialization, source preservation, and RAFT guidance, but no final composite;
5. **full pipeline:** primitive hard-warp initialization, RAFT guidance, source preservation, restoration, and final compositing.

The generated outputs and metadata are stored under `backend/results/core_ablation`. Each sample stores the raw diffusion image, pre-composite image, final image, manifest file, and diagnostic arrays for total energy, flow loss, color loss, noise norm, and guidance norm.

7.4 Quantitative Results

Table 7.1 summarizes the mean over three seeds. Lower values are better for flow loss, total energy, and background preservation error. Higher sharpness is generally preferable, but it must be interpreted with the visual result because compositing can produce a sharp object while also increasing boundary contrast.

The main numerical result is that RAFT guidance improves the pre-composite diffusion process. Compared with the no-RAFT diffusion variant, RAFT reduces the final flow loss from 8.46 to 5.80, a reduction of approximately 31 percent. It reduces the best recorded flow loss from 8.16 to 3.07, a reduction of approximately 62 percent. It also reduces the final total guidance energy from 30.57 to 21.86, a reduction of approximately 29 percent. Background preservation error improves from 0.0136 to 0.0080 in the diffusion-only comparison.

These numbers support the claim that RAFT guidance contributes measurable optimization pressure during diffusion. They do not prove that RAFT is solely responsible for the final visually best apple image, because the final full-pipeline result applies deterministic compositing.

7.5 Diagnostic Trajectories

Figure 7.1 plots the recorded RAFT flow loss over logged guidance steps. The RAFT-guided diffusion variants consistently descend below the no-RAFT variant. The original file-flow baseline has higher final flow loss and more instability near the end of sampling. The full pipeline follows the same guided-diffusion behavior as the RAFT variant before its final compositing stage.

Figure 7.2 shows the corresponding total guidance energy. The same pattern appears: RAFT-guided primitive diffusion reaches lower energy than no-RAFT diffusion. These curves are the strongest evidence that RAFT contributes to the diffusion trajectory itself.

7.6 Visual Results

Figure 7.3 shows final saved outputs for all variants and seeds. Rows correspond to seeds 0, 1, and 2. Columns show warp-inpaint-only, original Motion Guidance, diffusion without RAFT, diffusion with RAFT, and the full pipeline. The full pipeline produces the most visually coherent final apple across the three seeds. However, the deterministic warp-inpaint-only output is also visually strong, and both of these variants apply final compositing.

Figure 7.4 isolates the pre-composite diffusion outputs for the primitive diffusion variants. This figure is more appropriate for judging the diffusion stage itself. The no-RAFT condition preserves a partial object but does not convincingly complete the edit. RAFT-guided outputs generally align better with the intended motion according to the metrics, but they remain visually incomplete in some seeds. This supports a careful interpretation: RAFT helps the optimization objective, but restoration and compositing are still needed for the final contained-translation result.

7.7 Answer to the RAFT Contribution Question

The most important result is the distinction between optimization contribution and final rendering contribution. RAFT-guided diffusion measurably improves the intermediate diffusion process: it reduces final flow loss, best flow loss, total guidance energy, and background preservation error relative to the no-RAFT diffusion variant. Therefore, RAFT is not merely decorative in the code; it changes the measured diffusion trajectory.

At the same time, the final visually best apple image is not produced by diffusion alone. For contained

Table 7.1: Apple ablation results averaged over three seeds. The deterministic baseline has no diffusion loss trajectory because sampling is skipped.

Variant	Final source	Total energy	Flow loss	Best flow	Preserve L1	Runtime
Warp-inpaint only	Composite	–	–	–	0.0008	2.84 s
Original Motion Guidance	Diffusion	48.74	15.40	9.48	0.0065	170.71 s
Diffusion, no RAFT	Diffusion	30.57	8.46	8.16	0.0136	173.83 s
Diffusion + RAFT	Diffusion	21.86	5.80	3.07	0.0080	175.67 s
Full pipeline	Composite	21.71	5.67	3.13	0.0008	176.11 s

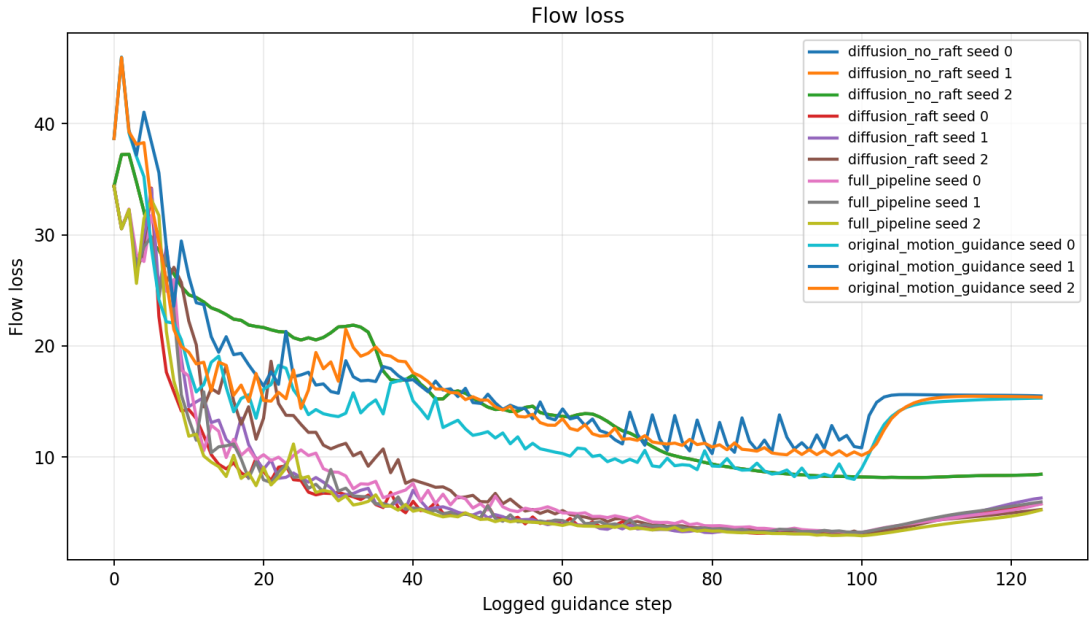


Figure 7.1: Recorded flow-loss trajectories for the apple ablation. RAFT-guided primitive diffusion reaches lower flow loss than the no-RAFT variant across the three seeds.

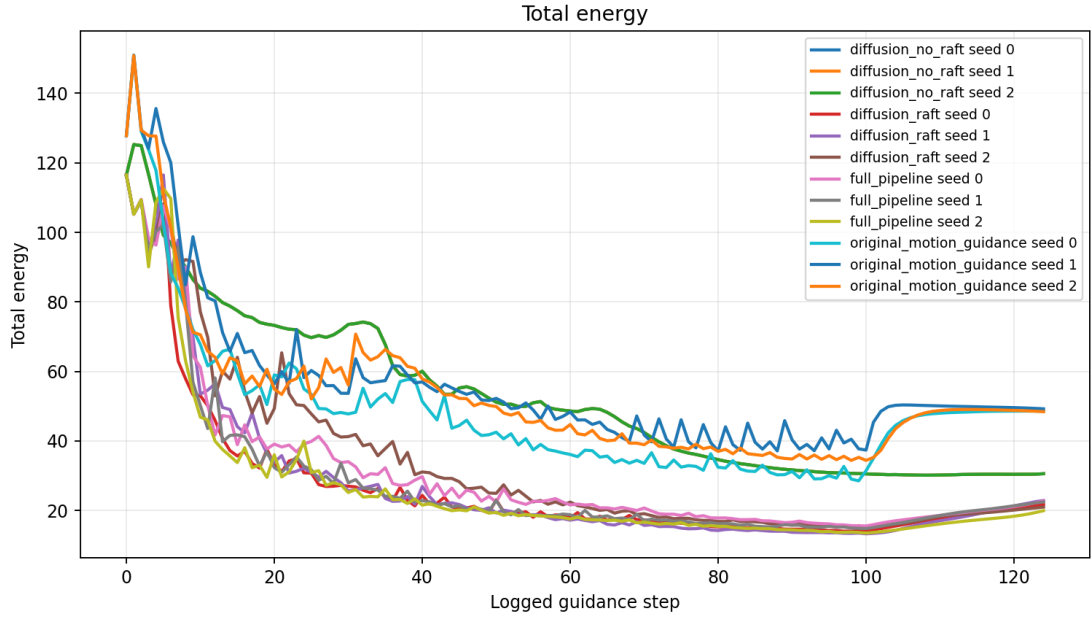


Figure 7.2: Total guidance-energy trajectories for the apple ablation. RAFT-guided variants have lower final energy than the no-RAFT diffusion condition.



Figure 7.3: Final saved apple outputs. Rows are seeds 0–2. Columns are warp-inpaint-only, original Motion Guidance, diffusion without RAFT, diffusion with RAFT, and full pipeline. The full pipeline is visually strong, but its final image is composited.



Figure 7.4: Pre-composite apple outputs for primitive diffusion variants. Rows are seeds 0–2. Columns are no-RAFT diffusion, RAFT-guided diffusion, and full-pipeline pre-composite output. These images separate diffusion behavior from final compositing.

translation, the full pipeline uses deterministic source-object compositing after diffusion. The final image therefore demonstrates the complete hybrid renderer, not an isolated diffusion result. The defensible claim is:

RAFT guidance improves the pre-composite diffusion trajectory. Deterministic restoration and compositing produce the cleanest final contained-translation image.

7.8 Additional Generated Cases

After the apple ablation, two additional real generated cases were run with three seeds each. These runs do not replace the controlled apple ablation, because they do not isolate RAFT versus no-RAFT conditions. They are included to address the concern that the thesis contained only one inspectable result and to show how the implementation behaves outside the main apple translation scene.

The teapot case uses primitive downward translation with final compositing enabled. It is the closest additional example to the apple setting. The tree case uses the available file-based squeeze flow and therefore saves the pre-composite diffusion result as the final image. It is better interpreted as a challenging non-rigid or file-flow stress case rather than as a polished translation result.

Figure 7.5 shows the teapot runs. The final images are stable across seeds because the contained-translation path applies restoration and source-object compositing after diffusion, as in the full apple pipeline.

Figure 7.6 shows the tree runs. Seed 2 is visually the most coherent. Seeds 0 and 1 show noticeable variation in crown shape and texture, which supports the thesis limitation that non-rigid or file-flow edits remain less reliable than contained rigid translations.

7.9 Status of Other Examples

Table 7.3 distinguishes the evaluated apple ablation and additional generated cases from configured or presentation examples. A configured preset is not counted as an evaluated result unless its complete output artifacts are available for inspection.

7.10 Evaluation Summary

The evaluation supports five conclusions. First, the complete pipeline can produce a stable contained rightward apple translation across three seeds. Second, RAFT-guided primitive diffusion improves the measured diffusion trajectory compared with no-RAFT diffusion. Third, original file-flow Motion Guidance performs worse than the primitive RAFT-guided variants on this apple setup. Fourth,

Table 7.2: Additional generated cases averaged over three seeds. These results broaden the qualitative evidence but are not controlled component ablations.

Case	Motion	Final source	Flow loss	Best flow	Preserve L1	Runtime
Teapot	Translate down	Composite	4.23 ± 0.14	3.09 ± 0.16	0.0005 ± 0.0000	170.86 ± 1.50 s
Tree	File-flow squeeze	Diffusion	3.65 ± 0.39	2.02 ± 0.14	0.0014 ± 0.0004	172.68 ± 3.28 s

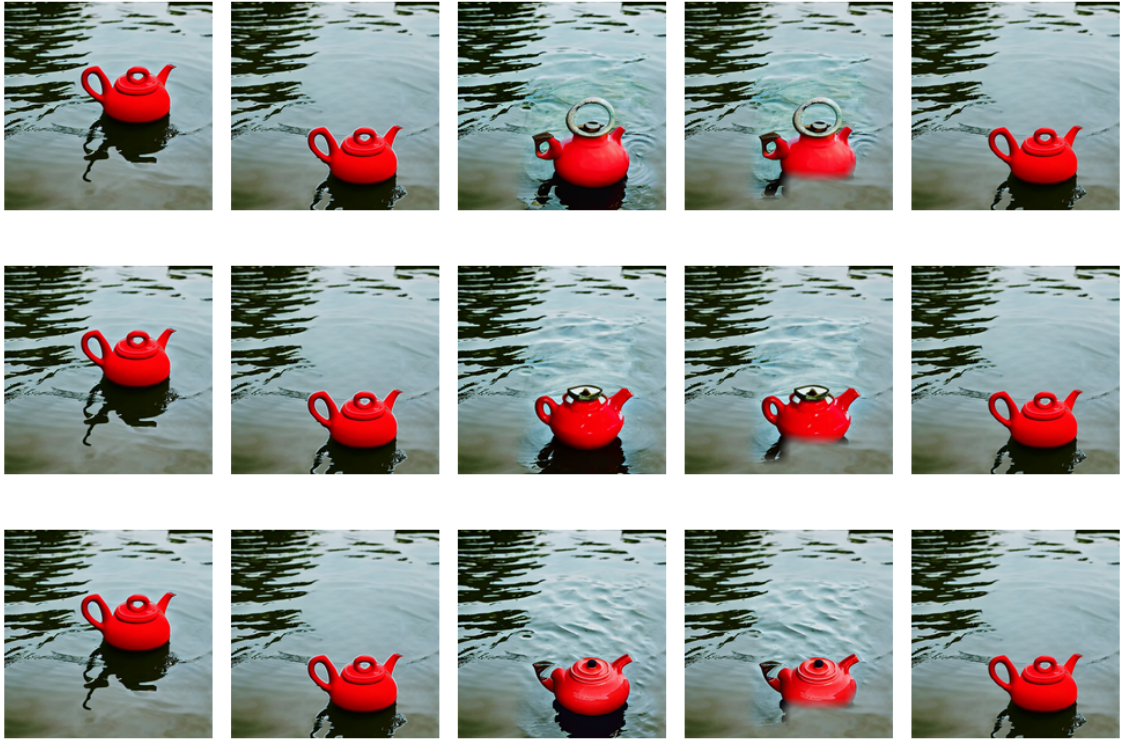


Figure 7.5: Additional teapot translation case over three seeds. Columns show source, warped initialization, decoded diffusion output, and final composited output.

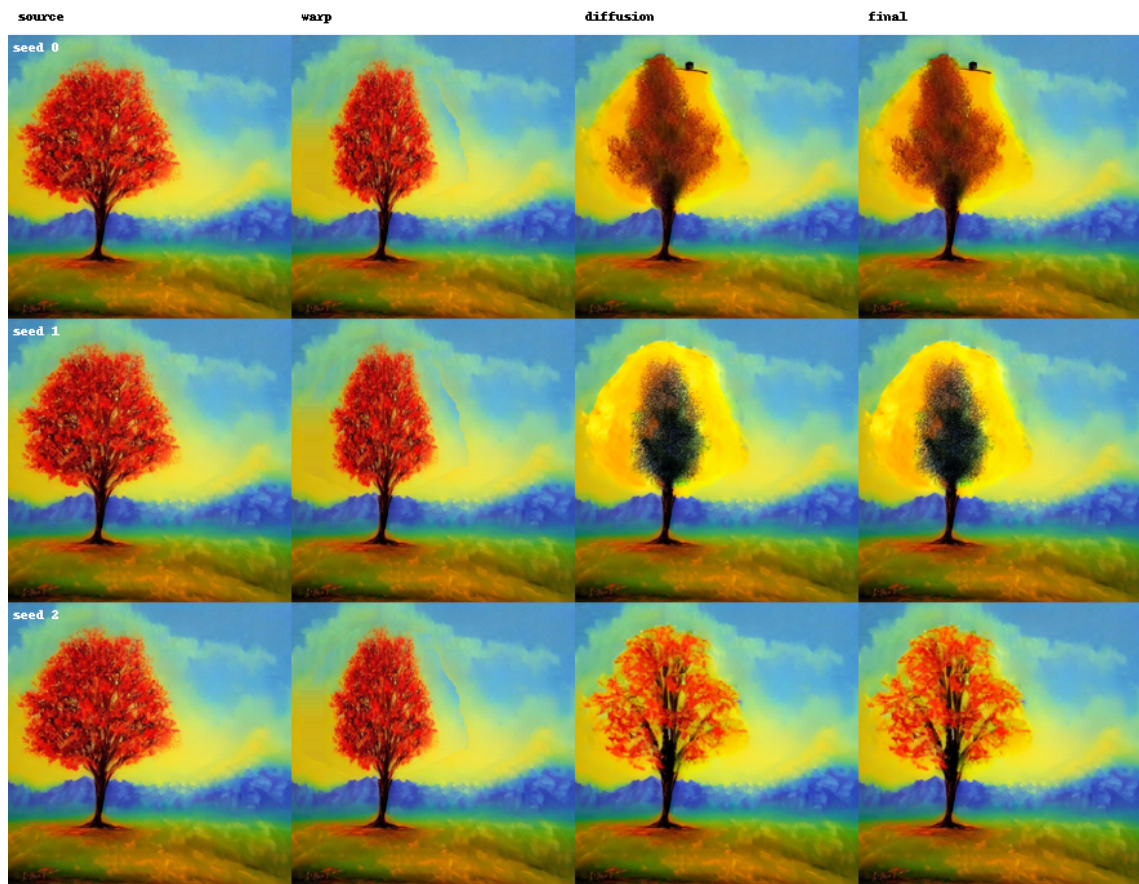


Figure 7.6: Additional tree file-flow squeeze case over three seeds. This case is useful as a stress test; it should not be presented as a polished rigid-translation result.

Table 7.3: Summary of examples and evidence status.

Example	Motion	Output Type	Evaluation Summary
Apple	Translate right	15-run ablation	Primary controlled quantitative and qualitative evidence.
Teapot	Translate down	3 generated seeds	Additional contained-translation evidence; final image uses compositing.
Tree	File-flow squeeze	3 generated seeds	Additional stress case; non-rigid/file-flow behavior is less stable.
Lion	Translate left	Smoke test only	Excluded from main results because the object support touches the image boundary and clips during translation.
Woman	Non-rigid	Shipped asset mode	Excluded from evaluation.
Topiary	Non-rigid	Shipped asset mode	Excluded from evaluation.

additional teapot and tree runs broaden the inspectable evidence, but they are not controlled ablations. Fifth, the final visual quality of contained translation is dominated by restoration and compositing, so the thesis must not claim that diffusion alone creates the final apple or teapot result.

The evidence remains narrow. The only full component ablation covers one object category, one background, one translation magnitude, one mask, one RAFT-iteration setting, and three seeds. The additional teapot and tree cases reduce the single-result weakness, but they do not establish broad generalization or state-of-the-art superiority.

Chapter 8

Component Analysis

8.1 Purpose of the Analysis

This chapter interprets the controlled apple ablation introduced in Chapter 7. The goal is not to prove broad generalization; the goal is to isolate what the implemented components contribute in the one complete, reproducible case study available in the thesis package.

The final editing pipeline can be written as

$$\hat{x} = \mathcal{C}(\mathcal{R}(\mathcal{D}_\phi(\mathcal{W}(x_0, F_\theta), F_\theta, M))), \quad (\text{Equation 8.1})$$

where x_0 is the input image, F_θ is the primitive target flow, M is the edit mask, $\mathcal{W}$ is geometric initialization, $\mathcal{D}_\phi$ is the motion-guided diffusion sampler, $\mathcal{R}$ is restoration, and $\mathcal{C}$ is final compositing. For contained translation, $\mathcal{C}$ can replace the decoded diffusion output in the saved final image.

8.2 Ablation Variants

The ablation compares five variants over three seeds. Table 8.1 summarizes their purpose. The important design choice is that the no-RAFT and RAFT variants both use the same primitive hard-warp initialization and both disable final compositing. This makes them the fairest pair for measuring the contribution of RAFT-guided diffusion.

8.3 Contribution of Primitive Flow and Hard-Warp Initialization

Primitive flow changes the interface from file-based target motion to parameterized motion. For translation, the target flow is

$$F_\theta(p) = (\Delta x, \Delta y), \quad (\text{Equation 8.2})$$

for pixels p inside the editable support. In the apple experiment, $\Delta x = 150$ and $\Delta y = 0$.

Hard-warp initialization then makes the large displacement explicit:

$$x_{\text{warp}}(p + F_\theta(p)) = x_0(p), \quad p \in M. \quad (\text{Equation 8.3})$$

This is why the deterministic warp-inpaint-only variant already produces a visually coherent moved apple. This result is useful, but it also creates an important interpretive boundary: the fact that the

final apple moves does not by itself prove that diffusion learned or inferred the displacement.

8.4 Contribution of RAFT-Guided Diffusion

The clearest evidence for RAFT is the comparison between *diffusion without RAFT* and *diffusion with RAFT*. Both variants use primitive hard-warp initialization, source preservation, the same prompt, the same DDIM step count, and the same seeds. The difference is the motion-guidance weight.

Across three seeds, adding RAFT guidance reduced final flow loss from 8.46 to 5.80 and reduced the best recorded flow loss from 8.16 to 3.07. It also reduced total guidance energy from 30.57 to 21.86 and improved background preservation error from 0.0136 to 0.0080.

This establishes a measured contribution:

$$\Delta L_{\text{flow, last}} = 8.46 - 5.80 = 2.66, \quad (\text{Equation 8.4})$$

which corresponds to approximately 31 percent lower final flow loss. For the best flow loss observed during sampling,

$$\Delta L_{\text{flow, min}} = 8.16 - 3.07 = 5.09, \quad (\text{Equation 8.5})$$

which corresponds to approximately 62 percent lower best flow loss.

Therefore RAFT guidance contributes to the diffusion trajectory. The saved diagnostic plots show this as a lower flow-loss and total-energy curve for RAFT-guided variants. However, the pre-composite images remain visually imperfect, so the contribution is best described as improved motion alignment rather than complete final rendering.

8.5 Contribution of Restoration and Compositing

After an object is translated, the source location becomes exposed:

$$H = M \setminus T_{\theta}(M), \quad (\text{Equation 8.6})$$

where H is the source-hole region and $T_{\theta}(M)$ is the transformed support. Restoration targets this region, while compositing preserves source-object texture at the target location.

The final full-pipeline image is visually stable across three seeds, but its final output source is recorded as *final composite*. The deterministic warp-inpaint-only variant also reaches a very low final preservation error of 0.0008 and a sharp object, despite skipping diffusion entirely. This shows that final appearance is strongly influenced by deterministic movement, restoration, and compositing.

This does not invalidate the RAFT result. It clarifies the role separation:

- RAFT-guided diffusion improves the measured pre-composite motion objective.
- Restoration and compositing produce the cleanest final contained-translation image.

8.6 Component Findings

The ablation supports the following component-level findings:

1. Primitive flow improves controllability by replacing dense manual flow specification with explicit motion parameters.
2. Hard-warp initialization enforces large rigid translation before diffusion begins.
3. RAFT guidance measurably lowers flow loss and total guidance energy during diffusion.
4. Restoration and compositing dominate the visual quality of the final contained apple output.
5. The complete pipeline is a hybrid renderer, not a pure diffusion-only method.

The strongest defensible answer to the ablation question is therefore not that RAFT alone produces the final apple. The stronger and more honest conclusion is that RAFT contributes measurable pre-composite alignment, while deterministic compositing is required for the final visually convincing rigid-translation result.

Table 8.1: Controlled apple ablation variants.

Variant	Diffusion	Final composite	Question answered
Warp-inpaint-only	No	Yes	How strong is the deterministic movement and compositing baseline?
Original Motion Guidance	Yes	No	How does the file-flow baseline behave on the apple setup?
Diffusion, no RAFT	Yes	No	What happens when hardwarp initialization is used without RAFT guidance?
Diffusion + RAFT	Yes	No	Does RAFT guidance improve the pre-composite diffusion trajectory?
Full pipeline	Yes	Yes	What does the complete user-facing method produce?

Chapter 9

Limitations, Ethics, and Future Work

9.1 Limitations

The proposed system exposes practical controls for motion-guided image editing, but the empirical evidence is still limited. The final evaluation includes a controlled apple ablation with five variants and three seeds, plus additional three-seed teapot and tree generated cases. This is stronger than a single qualitative result, but the only full component ablation remains one object, one background, one translation magnitude, one mask, and one main hyperparameter setting. The following limitations bound the conclusions that can be drawn.

9.1.1 Limited Reliability for Non-Rigid Motion

The current pipeline is designed most completely for simple geometric transformations. Non-rigid examples, such as hair growth or topiary deformation, involve changes in local shape, texture, identity, and boundary structure that a single primitive flow or simple post-processing rule may not capture. The woman and topiary presets can load shipped output assets, so they are presentation examples rather than generated evidence. The generated tree squeeze runs are included only as a file-flow stress case and show visible seed-dependent deformation. The thesis therefore draws no strong performance conclusion about reliable non-rigid deformation.

9.1.2 Source-Hole Restoration

When an object is moved, the original object location becomes newly exposed. This region, denoted as H , must be reconstructed as plausible background:

$$H = M \setminus T_{\theta}(M), \quad (\text{Equation 9.1})$$

where M is the original object mask and $T_{\theta}(M)$ is the transformed mask. Although restoration and compositing improve this region, the filled background may still show synthetic texture, weak boundary blending, or residual artifacts. This is the main visual limitation of the current lightweight pipeline.

The implementation supports LaMa and simpler filling strategies, but the thesis artifacts do not contain a controlled comparison among them. Large disocclusions, structured backgrounds, shadows, reflections, and repeated textures remain difficult regardless of the selected restorer.

9.1.3 Dependence on Masks, Flows, and Cached Latents

The pipeline depends on the quality of the edit mask, target flow, inversion latent, and cached DDIM latents. If the mask is inaccurate, the object boundary may be damaged or background pixels may be moved incorrectly. If the flow is inconsistent with the intended edit, the guidance loss may push the image toward an undesirable deformation.

This dependency also affects reproducibility. A complete runnable example requires the input image, mask, target flow or primitive parameters, inversion latent, and cached sampling latents. For example, the cat image available in the repository is a presentation asset, but the corresponding complete runnable dataset is not available. Therefore, it should not be reported as a completed experiment.

9.1.4 Limited Quantitative Evaluation

The evaluation reports quantitative metrics for the apple ablation and for additional teapot and tree runs, but it is not equivalent to a benchmark evaluation and cannot establish broad statistical reliability, average improvement, or superiority over competing methods. More rigorous evaluation requires repeated trials across object categories, motion magnitudes, mask qualities, backgrounds, random seeds, prompts, and RAFT iteration counts.

The reported apple ablation metrics are useful for the central RAFT question because they compare no-RAFT and RAFT-guided diffusion under fixed seeds. The teapot and tree metrics are useful as additional inspectable cases, but they should not be interpreted as proof of broad generalization.

9.1.5 Internal and External Validity

The final apple ablation was rerun with fixed seeds and retained intermediate outputs, so the RAFT versus no-RAFT comparison is stronger than the earlier development notes. However, external validity is still limited because the controlled runs cover only one scene, while the teapot and tree cases are supplementary rather than controlled ablations. Finally, the contained-translation renderer replaces the decoded diffusion sample with a deterministic source-object composite in the full pipeline, so the final apple and teapot images do not isolate the contribution of diffusion refinement. The thesis therefore interprets RAFT as improving pre-composite diffusion alignment rather than as solely producing the final image.

9.2 Ethical Considerations

Motion-guided image editing can be used for creative design, visual prototyping, and research demonstrations, but it can also be misused for deceptive manipulation. The most important ethical require-

ment is disclosure. Edited images should not be presented as unmodified photographs when the edit changes object position, human appearance, or scene meaning.

Human-image examples require additional care. Edits involving faces, bodies, hair, clothing, or identity can affect consent, privacy, and representation. In this thesis, the woman example is treated cautiously and is not used to make strong algorithmic claims. This avoids overstating performance on human identity-sensitive editing.

The system should also include clear user-facing boundaries in future deployment. Recommended safeguards include edit provenance metadata, visible warnings for generated outputs, restrictions on harmful impersonation, and careful dataset licensing. These safeguards are especially important if the method is extended beyond academic demonstration.

9.3 Future Work

The most important future direction is stronger source-hole restoration. A dedicated learned inpainting stage, better boundary harmonization, and shadow-aware compositing could improve realism at the original object location. This would directly address the main visual weakness observed in rigid translation examples.

Second, the method should be extended to more reliable non-rigid deformation. Instead of relying on a single flow field and generic diffusion guidance, future work could use part-aware masks, local deformation constraints, keypoint trajectories, or multi-region motion primitives. This would make examples such as hair growth, topiary deformation, and articulated object motion more defensible as algorithmic results.

Third, the evaluation should be expanded. A stronger experimental study should include multiple images per category, several displacement magnitudes, repeated random seeds, and quantitative comparison against baseline Motion Guidance, DragDiffusion, SDEdit, and simple warp-and-inpaint methods. The metrics proposed earlier in the thesis can be used to measure motion error, background preservation, source-hole plausibility, and boundary consistency.

Fourth, the user workflow can be improved through automatic mask generation, interactive drag controls, real-time preview, and automatic parameter validation. This would reduce the amount of manual setup needed before running an edit.

Finally, inference speed remains a practical limitation. The current system uses diffusion sampling, optical-flow guidance, and restoration, which can be expensive. Faster samplers, fewer refinement steps, model distillation, and asynchronous job scheduling would make the system more suitable for interactive use.

9.4 Summary

The correct scope is an implemented hybrid editing workflow with direct primitive controls, a focused fixed-seed apple ablation, and two additional generated case studies. The evidence shows RAFT-guided improvement in pre-composite diffusion alignment, while final visual quality depends heavily on restoration and compositing. Stronger restoration, reliable non-rigid editing, broader quantitative testing, and faster interaction remain open work.

Chapter 10

Conclusion

10.1 Conclusion

This thesis developed an inference-time ML extension of Motion Guidance for single-image editing. The implementation couples Stable Diffusion v1.4, AutoencoderKL, frozen CLIP text conditioning, a conditional U-Net, and pretrained RAFT optical flow. It adds primitive-generated supervision, spatial gradient weighting, latent preservation, geometric initialization, and restoration around this learned model stack.

The central research question was how Motion Guidance can expose more direct control without training a new diffusion model. The implemented answer is a staged pipeline: user parameters are converted into target flows, geometric initialization enforces displacement, diffusion guidance remains available during sampling, and restoration plus compositing render the contained rigid-translation result.

10.2 Summary of Contributions

The thesis makes five ML-oriented implementation contributions. First, it converts translation, scale, stretch, and rotation into dense target labels for RAFT guidance. Second, it differentiates flow and color losses through RAFT and the VAE decoder to the DDIM latent. Third, it adds mask-dependent gradient weighting, clipping, scheduling, and source-latent replacement. Fourth, it encodes geometrically initialized images as alternative starting states for diffusion inference. Fifth, it records per-step loss and norm diagnostics for analysis of the guided sampler.

These are inference-time ML contributions rather than model-training contributions. The thesis does not introduce new network weights; it shows how pretrained generative and discriminative models can be composed, differentiated, and controlled for object-level editing.

10.3 Main Findings

Development records indicate that primitive flow alone did not reliably produce the requested large displacement. Geometric initialization resolves this functional problem by placing the object at the intended location explicitly. The final apple ablation confirms that deterministic movement and compositing are strong contributors to final visual quality.

The second finding is quantitative: RAFT-guided diffusion improves the pre-composite diffusion tra-

jectory. Across three apple seeds, adding RAFT guidance reduced final flow loss from 8.46 to 5.80, best recorded flow loss from 8.16 to 3.07, and total guidance energy from 30.57 to 21.86 compared with the no-RAFT diffusion variant. This directly answers the concern that RAFT might not contribute measurable evidence.

The third finding is architectural: learned motion guidance and source-hole completion are different problems. RAFT can supervise where content should move, but newly exposed background requires an inpainting or restoration model. The final path therefore separates ML-guided sampling from disocclusion handling. The visually strongest final apple and teapot translation results are hybrid composites, so those final images should not be presented as the output of diffusion alone.

The fourth finding is about scope. Additional teapot and tree runs reduce the weakness of having only one inspectable output. Teapot supports the practical contained-translation workflow, while tree shows that file-flow or non-rigid deformation remains less stable and should be treated as a stress case.

10.4 Answer to the Research Question

The thesis answers the research question by showing that Motion Guidance can be extended into a more controllable image-editing pipeline through a combination of explicit motion control and post-diffusion correction. The final system improves practical usability in three ways:

1. it converts simple geometric motion into dense supervision for a learned flow estimator;
2. it injects differentiable RAFT and appearance losses into latent DDIM inference;
3. it preserves source latents outside the edit region and supports stronger latent initialization.

This makes motion specification more direct than selecting a prepared dense flow file, while retaining compatibility with the baseline guidance machinery.

The refined answer to the central examination question is that RAFT guidance contributes to the diffusion stage by lowering measured flow loss and total energy, while deterministic restoration and compositing contribute most strongly to the final visual cleanliness of the contained apple translation.

10.5 Final Remarks

The work does not establish broad comparative superiority, statistical reliability across many scenes, or reliable non-rigid editing quality. Source-hole restoration can remain imperfect, and the final contained-translation images are deterministic composites rather than isolated diffusion outputs. The supported contribution is therefore the design, implementation, and focused fixed-seed evaluation of an auditable hybrid workflow.

Overall, the project shows how a latent diffusion generator and a neural optical-flow estimator can interact through inference-time gradients. Future work should extend the current diagnostics to more images, compare RAFT iterations and guidance hyperparameters, improve restoration comparisons, isolate pre-composite diffusion outputs in additional cases, and test learned restoration and non-rigid supervision.

References

- [1] D. Geng and A. Owens, “Motion guidance: Diffusion-based image editing with differentiable motion estimators,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=WIAO4vbnNV>
- [2] Z. Teed and J. Deng, “RAFT: Recurrent all-pairs field transforms for optical flow,” in *European Conference on Computer Vision*, 2020, pp. 402–419. [Online]. Available: <https://arxiv.org/abs/2003.12039>
- [3] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>
- [4] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=St1giarCHLP>
- [5] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10 684–10 695. [Online]. Available: <https://arxiv.org/abs/2112.10752>
- [6] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv preprint arXiv:2207.12598*, 2022. [Online]. Available: <https://arxiv.org/abs/2207.12598>
- [7] C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, and S. Ermon, “SDEdit: Guided image synthesis and editing with stochastic differential equations,” in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=aBsCjcPu_tE
- [8] A. Hertz, R. Mokady, J. Tenenbaum, K. Aberman, Y. Pritch, and D. Cohen-Or, “Prompt-to-prompt image editing with cross attention control,” *arXiv preprint arXiv:2208.01626*, 2022. [Online]. Available: <https://arxiv.org/abs/2208.01626>
- [9] T. Brooks, A. Holynski, and A. A. Efros, “Instructpix2pix: Learning to follow image editing instructions,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 18 392–18 402. [Online]. Available: <https://arxiv.org/abs/2211.09800>
- [10] X. Pan, A. Tewari, T. Leimkühler, L. Liu, A. Meka, and C. Theobalt, “Drag your GAN: Interactive point-based manipulation on the generative image manifold,” in *ACM SIGGRAPH 2023 Conference Proceedings*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.10973>

- [11] Y. Shi, C. Xue, J. Pan, W. Zhang, V. Y. F. Tan, and S. Bai, “Dragdiffusion: Harnessing diffusion models for interactive point-based image editing,” *arXiv preprint arXiv:2306.14435*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.14435>
- [12] A. Lugmayr, M. Danelljan, A. Romero, F. Yu, R. Timofte, and L. Van Gool, “RePaint: Inpainting using denoising diffusion probabilistic models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 461–11 471. [Online]. Available: <https://arxiv.org/abs/2201.09865>
- [13] R. Suvorov, E. Logacheva, A. Mashikhin, A. Remizova, A. Ashukha, A. Silvestrov, N. Kong, H. Goka, K. Park, and V. Lempitsky, “Resolution-robust large mask inpainting with fourier convolutions,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 2149–2159. [Online]. Available: <https://arxiv.org/abs/2109.07161>

Appendix A

Development and Weekly Progress Log

This appendix summarizes the development trail of the project. It is included as supporting evidence for the implementation process, experimental decisions, and final scope of the thesis. The main thesis chapters describe the method and evaluation; this appendix records how the final pipeline was reached.

A.1 Development Objective

The project objective was to build a controllable extension of Motion Guidance for image editing. The original baseline uses a source image x_0 , edit mask M , text condition c , and target flow F . The thesis extension adds:

1. explicit motion primitives for user-defined control;
2. selective refinement inside and outside the edit region;
3. hard warp initialization for stronger object displacement;
4. restoration and sharp compositing for source-hole cleanup;
5. a backend and frontend interface for repeatable demonstrations.

A.2 Technical Development Timeline

A.3 Main Code Artifacts

A.4 Experimental Progression

The development process followed a staged experimental progression. The baseline established that the Motion Guidance pipeline could be reproduced. Motion primitives then made the target motion easier to specify, but early experiments showed that primitive flow used only as a soft guidance signal was not strong enough for reliable displacement.

Hard warp initialization was the key turning point. By moving the object geometrically before diffusion refinement, the method changed the role of the diffusion sampler from performing the full motion to refining an already displaced image. This made rigid object translation practical.

Table 10.1: Weekly development summary.

Stage	Work Completed	Technical Detail	Outcome
Weeks 1–3	Baseline reproduction	Original Motion Guidance pipeline with fixed source image, mask, flow, and prompt	Established reproducible comparison point
Weeks 4–5	Motion primitives and selective refinement	Generated target flow using $F(p) = T(p) - p$ and weighted guidance using γ_{in} and γ_{out}	Primitive control improved usability, but flow-only guidance was weak
Weeks 6–7	Hard warp initialization	Applied $x_{\text{warp}}(p) = x_0(p - F(p))$ before diffusion refinement	Object translation became operational
Weeks 8–9	Background cleanup and final evaluation organization	Tested source-hole cleanup, compositing, and stage-wise comparison	Best early apple result selected and comparison structure prepared
Week 10	Thesis writing and restoration module cleanup	Organized method, contribution, limitation, and evaluation narrative	Thesis structure prepared
Week 11	LaMa restoration and presets	Added restoration options, example presets, validation, and cancel endpoint	Contained-translation path completed
Week 12	Before-after interface and presentation assets	Added source preview, server guards, and shipped-asset loading for selected presets	Presentation path separated from generated evidence

Table 10.2: Implementation artifacts used in the thesis.

Component	File
Motion primitive generation	backend/motion_primitives.py
Selective gradient weighting	backend/selective_refinement.py
Modified DDIM guidance loop	backend/lDM/models/diffusion/ddim_with_grad.py
Main generation pipeline	backend/generate.py
Background restoration and compositing	backend/background_restoration.py
Backend API	backend/mg_api/main.py
Frontend interface	frontend/src/app/page.tsx
Example commands	Commands

Restoration experiments were then added because object movement exposes the original object location. Several strategies were tested, including local filling, directional texture filling, patch-based restoration, OpenCV Telea inpainting, and LaMa-based restoration. The final system uses restoration and sharp compositing to improve the source-hole region while preserving the moved object.

A.5 Final Example Status

A.6 Development Summary

The final implementation was reached through a sequence of technical decisions. Primitive control made motion specification more direct, spatial weighting localized the guidance gradient by construction, geometric initialization enforced rigid motion, and restoration with source-object compositing completed the contained-translation renderer. The retained artifacts do not isolate the quality gain from each stage, so controlled ablation and broader evaluation remain future work.

Table 10.3: Final status of project examples.

Example	Motion Type	Final Status	Thesis Role
Apple	Primitive translation	Strong generated example	Main evidence for final pipeline
Teapot	Primitive translation	Strong generated example	Additional rigid-translation evidence
Lion	Primitive translation	Supported but less central	Secondary primitive example
Tree	Primitive stretch	Supported, less stable	Demonstrates broader primitive support
Woman	File flow with reference stabilization	Demonstration only	Human-image and non-rigid limitation case
Topiary	File flow with reference stabilization	Demonstration only	Non-rigid deformation limitation case
Cat	Presentation asset only	Not runnable as a complete experiment	Excluded from final evaluation

Appendix B

Commands, Parameters, and Reproducibility

This appendix lists the practical information needed to reproduce the main experiments. The commands are based on the submitted repository structure and the project command file. Some external assets, such as pretrained model checkpoints and the original Motion Guidance data archive, must be downloaded separately.

B.1 Environment

The backend follows the Motion Guidance environment with additional dependencies for the API and restoration stage. The original environment file specifies Python 3.8.5, PyTorch 1.11.0, torchvision 0.12.0, CUDA toolkit 11.3, and the original Motion Guidance dependencies. The updated requirement files also include FastAPI, Uvicorn, Python multipart support, OpenCV, and optional LaMa restoration through `simple-lama-inpainting`.

The basic setup is:

```
cd motion_guidance_repo/backend
conda env create -f environment.yml
conda activate motion_guidance
```

If the updated pip requirements are used, install them inside the activated environment:

```
pip install -r requirements.txt
```

The original Motion Guidance setup also requires the RAFT checkpoint and the Stable Diffusion v1.4 checkpoint. The data folder must contain the input image, masks, flows, inversion latent, and cached latents for each runnable example.

B.2 Required Repository Structure

For a runnable example, the following files are expected:

```
backend/data/<example>/pred.png
backend/data/<example>/start_zt.pth
backend/data/<example>/flows/<target_flow>.pth
backend/data/<example>/flows/<mask>.pth
backend/data/<example>/latents/
```

The exact cached-latent structure depends on the original Motion Guidance data package. If cached latents are not available, the run configuration must be changed accordingly, for example by disabling `--use_cached_latents` and using the available initialization options.

B.3 Backend API

The backend API can be started from the backend directory:

```
cd motion_guidance_repo/backend
conda activate motion_guidance
uvicorn mg_api.main:app --host 0.0.0.0 --port 8000
```

The API exposes the project presets and generation endpoint used by the frontend. During development, an existing server process was stopped before restarting Uvicorn.

B.4 Frontend

The frontend is implemented with Next.js. It is located in:

```
motion_guidance_repo/frontend
```

The frontend provides example presets, input preview, before-after display, primitive controls, and backend job interaction. It should be run while the backend API is available on port 8000.

B.5 Main Example Parameters

The documented default apple-ablation values were:

```
ddim_steps = 120
scale = 7.5
guidance_weight = 30
clip_grad = 60
num_recursive_steps = 1
raft_iters = 1
precision = fp32
selective_inner_weight = 1.0
selective_outer_weight = 0.15
```


B.6 Core Apple Ablation Commands

The final apple ablation commands are generated from the backend directory:

```
cd motion_guidance_repo/backend
conda activate motion_guidance
```

```
python experiments/run_core_ablation.py \
  --results-root results/core_ablation \
  --seed 0 \
  --ddim-steps 120 \
  --raft-iters 1
```

```
python experiments/run_core_ablation.py \
  --results-root results/core_ablation \
  --seed 1 \
  --ddim-steps 120 \
  --raft-iters 1
```

```
python experiments/run_core_ablation.py \
  --results-root results/core_ablation \
  --seed 2 \
  --ddim-steps 120 \
  --raft-iters 1
```

Each generated command should be run from the backend directory. After all 15 runs complete, metrics and diagnostic plots are regenerated with:

```
python experiments/compute_metrics.py \
  --results-root results/core_ablation \
  --csv results/core_ablation/metrics.csv \
  --json results/core_ablation/metrics.json
```

```
python experiments/plot_diagnostics.py \
  --results-root results/core_ablation \
  --output-dir results/core_ablation/diagnostic_plots
```

The final CSV file used in Chapter 7 is:

```
backend/results/core_ablation/metrics.csv
```

B.7 Direct Backend Command

The teapot command below is the clearest direct backend run for the final pipeline:

```
python generate.py \  
  --prompt "a teapot floating in water" \  
  --input_dir ./data/teapot \  
  --edit_mask_path down150.mask.pth \  
  --target_flow_mode primitive \  
  --target_flow_name down150.pth \  
  --primitive_kind translate \  
  --primitive_dx 0 \  
  --primitive_dy 150 \  
  --use_hard_warp_init \  
  --use_selective_refinement \  
  --selective_inner_weight 1.0 \  
  --selective_outer_weight 0.15 \  
  --use_cached_latents \  
  --log_freq 5 \  
  --ddim_steps 120 \  
  --scale 7.5 \  
  --num_recursive_steps 1 \  
  --guidance_weight 30 \  
  --clip_grad 60 \  
  --raft_iters 1 \  
  --precision fp32 \  
  --disable_dataparallel \  
  --save_dir ./results/teapot.down150.lama
```

The same command structure can be adapted for apple by changing the prompt, input directory, mask name, flow name, translation values, and save directory.

B.8 App Presets

The following preset values were used in the frontend:

Apple:

prompt: an apple on a wooden table

```
input_dir: ./data/apple
mask: right.mask.pth
target_flow: right.pth
primitive: translate
dx: 150
dy: 0
```

Teapot:

```
prompt: a teapot floating in water
input_dir: ./data/teapot
mask: down150.mask.pth
target_flow: down150.pth
primitive: translate
dx: 0
dy: 150
```

Woman:

```
prompt: a portrait photo of a woman
input_dir: ./data/woman
mask: growHair.mask.pth
target_flow: growHair.pth
mode: precomputed flow
```

Topiary:

```
prompt: a photo of topiary
input_dir: ./data/topiary
mask: mask.pth
target_flow: flow.pth
mode: precomputed flow
```

B.9 Cat Example Status

The cat example is not included as a completed runnable experiment. The repository contains:

```
backend/assets/cat.png
```

This file is a presentation asset, not a complete Motion Guidance input folder. A runnable cat experiment would require:

backend/data/cat/pred.png
backend/data/cat/start_zt.pth
backend/data/cat/flows/right150.mask.pth
backend/data/cat/flows/right150.pth
backend/data/cat/latents/

Until those files exist, cat should be reported only as unavailable, not as an evaluated result.

B.10 Reproducibility Notes

Reproducibility depends on four conditions. First, the same model checkpoints and example data must be available. Second, the mask and target flow must match the example configuration. Third, the same sampling parameters should be used. Fourth, the same restoration option should be enabled when comparing final outputs.

The thesis therefore treats the apple ablation as its directly inspectable case study. Teapot is a reproducible configuration when all model and data dependencies are available, but its output is not included in the final metric table. Woman, topiary, and cat are excluded from evaluation for the reasons stated above.

Table 10.4: Main reproducibility parameters.

Example	Prompt	Input Dir.	Mode	Key Parameters
Apple	an apple on a wooden table	./data/apple	primitive translate	$d_x = 150, d_y = 0$, outer weight 0.15
Teapot	a teapot floating in water	./data/teapot	primitive translate	$d_x = 0, d_y = 150$, outer weight 0.15
Lion	a photo of a lion	./data/lion	primitive translate	$d_x = -100, d_y = 0$, outer weight 0.15
Tree	a painting of a lone tree	./data/tree	primitive stretch or file flow	stretch support; less stable than translation
Woman	a portrait photo of a woman	./data/woman	presentation mode	asset excluded from evaluation
Topiary	a photo of topiary	./data/topiary	presentation mode	asset excluded from evaluation

Appendix C

Additional Result Images

This appendix provides additional qualitative material used to support the evaluation in Chapter 7. The appendix does not introduce new claims; it documents the visual evidence available in the thesis folder and clarifies the status of examples whose complete result images are not included in the local LaTeX project.

C.1 Apple Pipeline Result

The apple example is the clearest visual support for the proposed primitive-translation pipeline. It combines explicit displacement, hard warp initialization, background restoration, and sharp compositing.

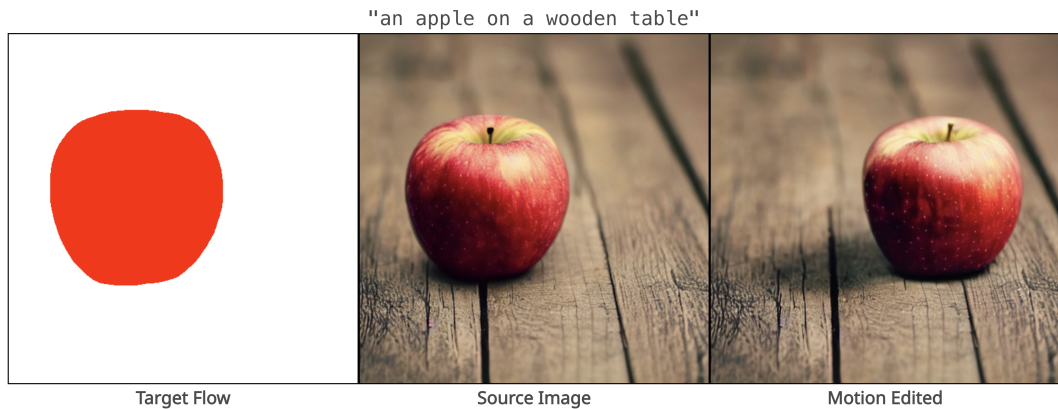


Figure 10.1: Apple result asset used as the main qualitative evidence for controllable rigid object translation.

C.2 Apple Ablation Grids

The final apple ablation contains five variants over three seeds. Figure 10.2 shows the final saved image for each run. Rows correspond to seeds 0, 1, and 2. Columns correspond to warp-inpaint-only, original Motion Guidance, diffusion without RAFT, diffusion with RAFT, and the full pipeline. Figure 10.3 shows pre-composite diffusion outputs for the primitive diffusion variants. It is included because it separates diffusion behavior from final deterministic compositing.

C.3 Additional Generated Cases

Figure 10.4 shows the additional teapot translation case. This example uses the same contained-translation path as the full apple pipeline, so the final column is a hybrid output that combines diffusion-stage processing with deterministic restoration and source-object compositing.

Figure 10.5 shows the additional tree file-flow squeeze case. This case is included as a stress example. Seed 2 is the cleanest representative output, while seeds 0 and 1 show stronger deformation and texture variation.

C.4 Excluded Presentation Assets

The repository contains woman and topiary images used by an optional presentation mode. The application can load these assets directly after sampling, so they are deliberately omitted from this results appendix and do not support algorithmic claims.

C.5 Additional Example Status

Table 10.5 summarizes the status of the examples discussed in the thesis. It distinguishes included evidence, documented configurations, presentation assets, and unavailable examples.

C.6 Interpretation

The appendix contains the directly inspectable apple ablation outputs and the additional teapot and tree generated runs. The remaining presets do not add empirical evidence in the local package. Cat is also excluded because its complete input folder, flow, mask, inversion latent, and cached latents are unavailable.



Figure 10.2: Final outputs from the apple ablation. This grid supports the qualitative comparison in Chapter 7.

Table 10.5: Status of additional qualitative examples.

Example	Category	Image Availability	Use in Thesis
Apple	Primitive rigid translation	15 ablation runs included	Main quantitative and qualitative evidence
Teapot	Primitive rigid translation	Three generated seeds included	Additional contained-translation evidence
Tree	File-flow squeeze	Three generated seeds included	Stress case for non-rigid/file-flow behavior
Lion	Primitive translation	Smoke output only	Excluded from main evidence because translation clips at the image boundary
Woman	Shipped presentation asset	Omitted from results	Excluded from evaluation
Topiary	Shipped presentation asset	Omitted from results	Excluded from evaluation
Cat	Presentation asset only	Not a complete runnable dataset	Excluded from final evaluation



Figure 10.3: Pre-composite outputs for primitive diffusion variants. Rows are seeds 0–2; columns are no-RAFT diffusion, RAFT-guided diffusion, and the full-pipeline pre-composite output.

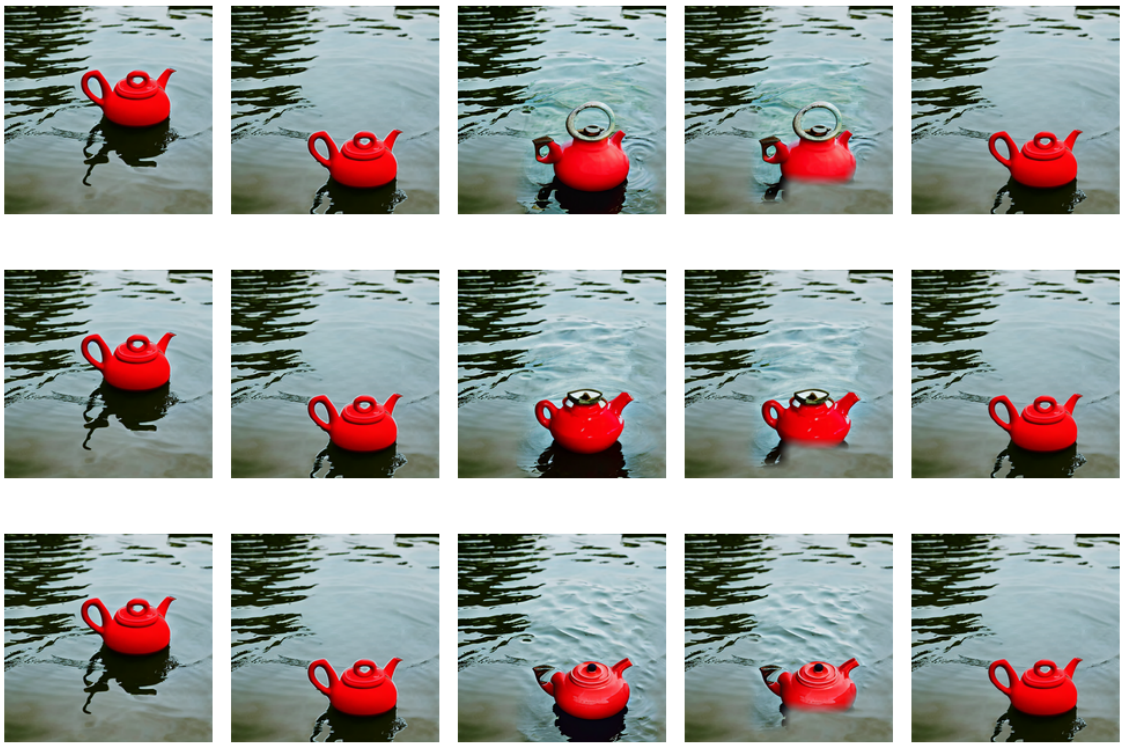


Figure 10.4: Additional teapot translation runs over seeds 0–2. Columns show source, warped initialization, decoded diffusion output, and final composited output.

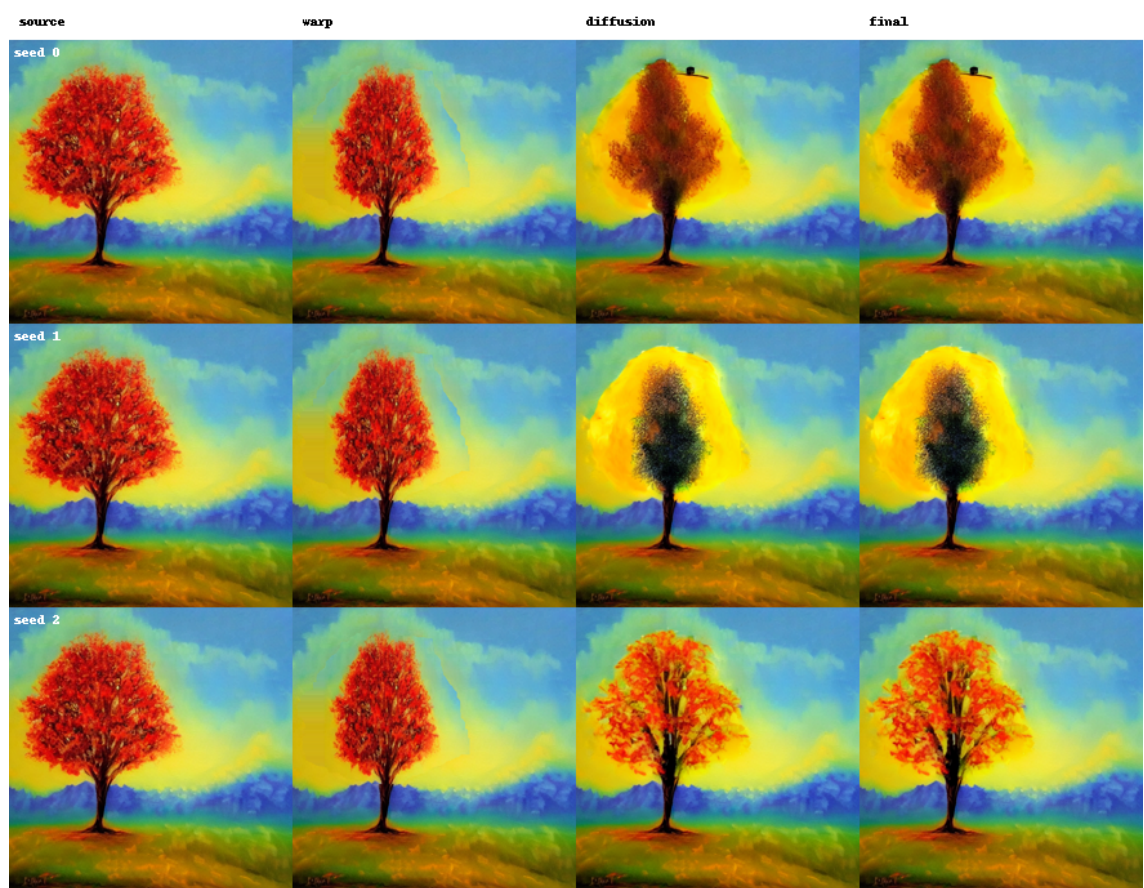


Figure 10.5: Additional tree file-flow squeeze runs over seeds 0–2. The case is useful for diagnosing non-rigid/file-flow behavior, not for claiming polished non-rigid editing.